



Forside

IS-314 : 2026

Tittel: Context-Aware AI Assistant for Municipal Caseworkers

Emnekode:	IS-314
Emnenavn:	Bacheloroppgave i IT og informasjonssystemer
Emneansvarlig:	Hallgeir Nilsen
Veileder:	Rania Fahim Hassan Ibrahim Elgazzar
Oppdragsgiver:	Rune Ødegård og Dagfinn Øksendal

Studenter:

Etternavn	Fornavn
Tamm	Kata-Loore
Hovet	Frank
Strand	Stine
Aarebakk	Jon Engravslia
Ege	Jørgen

Jeg/vi bekrefter at vi ikke siterer eller på annen måte bruker andres arbeider uten at dette er oppgitt, og at alle referanser er oppgitt i litteraturlisten.	JA	NEI
Kan besvarelsen brukes til undervisningsformål?	JA	NEI
Vi bekrefter at alle i gruppa har bidratt til besvarelsen	JA	NEI



Table of Contents

1. Introduction.....	1
1.1 The Client.....	1
1.2 Problem Domain.....	2
1.3 Project Description.....	2
1.4 Scope and Limitations.....	3
1.5 Technologies Used.....	3
2. Methodology and Process.....	4
2.1 SAFe and the FoU-2026 Program.....	4
2.2 PI Planning.....	5
2.3 GitHub Projects.....	6
2.4 Roles and Responsibilities.....	6
2.5 Scrum and Internal processes.....	8
2.6 MoSCoW Prioritisation.....	8
3. Project Execution.....	9
3.1 Product Increment 1.....	9
3.1.1 PI 1 Commit.....	9
3.1.2 Sprint 1 - Analysis and UI Design.....	10
3.1.3 Sprint 2 - Frontend Prototype and System Architecture Design.....	14
3.1.4 Sprint 1 Reflection.....	16
3.1.5 Sprint 2 Reflection.....	17
3.1.6 PI 1 Review.....	17
3.2 Product Increment 2.....	18
3.2.1 PI 2 Commit.....	18
3.2.2 Sprint 1 - Establishing the Backend Foundation.....	19
3.2.3 Sprint 2 - Backend Architecture and Scope Adjustments.....	20
3.2.4 Sprint 1 Reflection.....	22
3.2.5 Sprint 2 Reflection.....	23
3.2.6 PI 2 Review.....	24
3.3 Product Increment 3.....	26
3.3.1 PI 3 Commit.....	26
3.3.2 Sprint 1 - Final Prioritisation and MVP Development.....	27
3.3.3 Sprint 1 Review.....	27
3.3.4 Remaining Work.....	28
4. The Final Product.....	29
4.1 Application Demo.....	29

4.2 System Architecture.....	29
4.3 Future Development.....	31
4.4.1 Retrieved-Augmented Generation (RAG).....	31
4.4.2 Direct Archive Integration.....	31
4.4.3 Optical Character Recognition (OCR).....	31
4.4.4 Web Tokens for Session Management.....	31
4.4.5 Specialized Agents and Workflow Orchestration.....	32
4.4.6 Conversation Compression.....	32
4.4.7 Structured Logging Tools.....	32
5. Reflection.....	32
5.1 Quality Assurance and Resource Management.....	33
5.2 What the Team Would Do Differently.....	35
5.3 Key Learning Outcomes.....	35
5.3.1 The Importance of Structure.....	36
5.3.2 The Importance of Resource Management.....	36
5.3.3 The Importance of Teamwork.....	36
6. Conclusion.....	37
References.....	39
Appendix.....	40
Appendix A: Project Description.....	40
A1: Project Description Part 1.....	40
A2: Project Description Part 2.....	40
Appendix B: Analysis.....	41
B1: Sprint 1 Interview Questions.....	41
B2: Sprint 2 Interview Questions.....	41
B3: Supervision & Illegality Caseworker Workflow and Pain Points.....	42
B4: Caseworker Persona 1.....	43
B5: Scenario 1 - Persona Workflow without Application.....	44
B6: Caseworker Persona 2.....	45
B7: Scenario 2 - Persona Workflow with Application.....	46
B8: User Journey.....	47
B9: Caseworker Feedback on User Journey.....	48
B10: MoSCoW Analysis Frontend.....	50
Appendix C: Design Material.....	51
C1: Initial UI Sketches.....	51
C2: UI Navigation and Planned Pages.....	54
C3: Login Page Wireframe.....	55
C4: Case Overview Page Wireframe.....	55
C5: Case Page Wireframe.....	55
C6: Chat Page Wireframe.....	56
C7: Settings Page Wireframe.....	56

C8: Profile Page Wireframe.....	57
C9: Create Case Page Wireframe.....	57
Appendix D: WinUI Prototypes.....	58
D1: Login Page Prototype.....	58
D2: Case Overview Page Prototype.....	58
D3: Case Page Prototype.....	58
D4: Chat Page Prototype.....	59
D5: Settings Page Prototype.....	59
D6: Profile Page Prototype.....	59
D7: Login Page Final.....	60
D8: Case Overview Final.....	60
D9: Case Page Final.....	61
D10: Chat Page Final.....	61
D11: Settings Page Final.....	62
D12: Profile Page Final.....	62
D13: Create Case Page Final.....	63
D14: Chat Overview Page Final.....	63
Appendix E: System Architecture and Flow.....	64
E1: First Iteration System Architecture.....	64
E2: Second Iteration System Architecture.....	65
E3: Third Iteration System Architecture.....	66
E4: Fourth Iteration System Architecture.....	67
E5: Fifth Iteration System Architecture.....	68
E6: Endpoint Architecture.....	69
E7: Sequence Diagram.....	69
Appendix F: Logging of Hours.....	70
F1: Timesheet PI 1.....	70
F2: Timesheet PI 2.....	72
F3: Timesheet PI 3.....	74
Appendix G: Self and Team Evaluation.....	75
G1: Self evaluation - Kata.....	75
G2: Self evaluation - Jon.....	75
G3: Self evaluation - Stine.....	76
G4: Self evaluation - Jørgen.....	76
G5: Self evaluation - Frank.....	77
G6: Team reflection.....	78
Appendix H: The Client's Evaluation.....	79
Appendix I: Group Contract.....	80

Figurelist

Figure 1: Organisational hierarchy.....	1
Figure 2: Proposed project description.....	2
Figure 3: Empty PI plan.....	5
Figure 4: Suggested DevOps roles.....	7
Figure 5: Initial DevOps role assignment.....	7
Figure 6: Committed PI 1 plan.....	9
Figure 7: Design thinking process.....	10
Figure 8: Planning and building caseworker workflow and pain points.....	11
Figure 9: Initial navigation map.....	12
Figure 10: CasePage wireframe.....	13
Figure 11: CasePage view.....	15
Figure 12: End of PI 1 plan.....	18
Figure 13: PI 2 plan.....	19
Figure 14: Orchestration and agentic system workflow.....	21
Figure 15: Early database schemas.....	22
Figure 16: End of PI 2 plan.....	25
Figure 17: PI 3 plan.....	26
Figure 18: Current database schemas.....	27
Figure 19: System architecture.....	29
Figure 20: Project Management Triangle.....	33

Preface

The team would like to express our sincere gratitude to Kristiansand Municipality for providing the opportunity to work on a project with real-world relevance for the public sector. In particular, we would like to thank Rune Ødegård and Dagfinn Øksendal, whose engagement, as well as willingness to share knowledge and provide guidance, has been invaluable throughout the project.

We would also like to thank our supervisor at the University of Agder, Rania Elgazzar, for her support and feedback throughout the project. Finally, we would like to thank the University of Agder for three years of education that have prepared us for this work. A special thanks goes to Hallgeir Nilsen and Geir Inge Hausvik for making this collaboration possible.

Summary

This bachelor's project is conducted as part of Kristiansand Municipality's planning and building department FoU-2026 research and development program, and sets out to develop a context-aware AI assistant for municipal building permit caseworkers. The report covers the team's project execution, along with the methodology and frameworks guiding three major working periods. Architectural decisions and further reflection are also included to highlight the research-driven and iterative nature of the project. The developed application is a full-stack system with a Windows desktop client (WinUI3 / C# / .NET) and a Python backend (FastAPI, FastMCP, Pydantic AI).

A demo video of the system is linked below:

<https://www.youtube.com/watch?v=bOFWErXhd3M>

1. Introduction

This bachelor's project is titled "Context-Aware AI Assistant for Municipal Caseworkers", though it is often simply referred to as "Local LLM". The project is conducted in collaboration with the Planning & Building Department in Kristiansand Municipality. The student group, nicknamed "Team Lambda" by the client, consists of a total of five members:

- Frank Hovet
- Jon Engravslia Aarebakk
- Jørgen Ege
- Kata-Loore Tamm
- Stine Strand

1.1 The Client

Kristiansand Municipality is amongst Norway's most digitally advanced municipalities, with a long-term commitment to research and development within artificial intelligence, data management and urban planning technology, as described in *Appendix A1: Project description part 1*. The client collaborates with public institutions such as Direktoratet for Byggkvalitet (DiBK), as well as other municipalities on topics such as applied AI use cases, including document interpretation, zoning plan accessibility and process automation.

Five core product owners are specified as shown in the hierarchy in *Figure 1*. The primary contacts from the client are Rune Ødegård, tech-lead and release train manager (RTE), and Dagfinn Øksendal, organisational coordinator. They serve as client representatives as well as project supervisors, providing guidance with regards to process and technical aspects throughout the project.

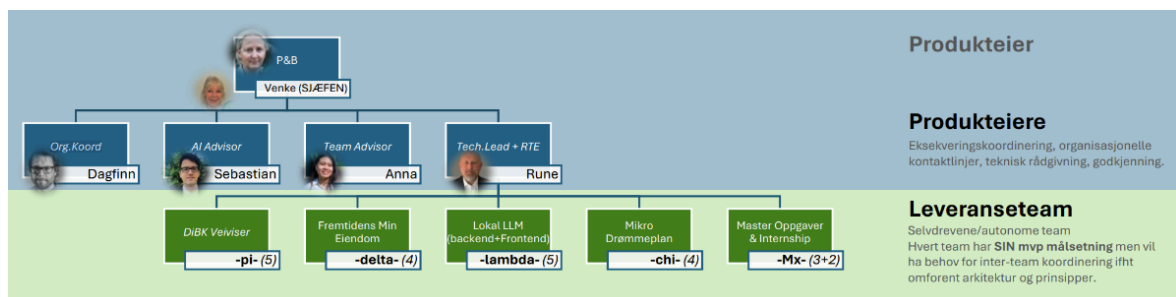


Figure 1: Organisational hierarchy

1.2 Problem Domain

The caseworkers within the department handle a high volume of complex building permit cases, each requiring navigation across multiple external systems to locate relevant regulations, precedents and planning documents. Currently, this process is time-consuming and requires a lot of manual work, forming the problem domain for the bachelor's project.

1.3 Project Description

At its core, the bachelor's project aims to explore the integration of AI assistants into the everyday workflow of municipal caseworkers. The assignment description provided by the product owners consists of two complementary parts. The first concerns developing a lightweight, context-aware desktop client application, capable of capturing the user's active work environment and relaying that context to an AI backend for assistance. The second part concerns designing and implementing a backend service that acts as the bridge between the client and the municipality's AI infrastructure, orchestrating communication with one or more large language models. Further details regarding the project description are included below in *Figure 2*.

Project description

The proposed project can be approached as a single integrated bachelor project or divided into **two complementary sub-projects** depending on student interests and available capacity.

Part 1: Context-Aware Client Application (Windows / Cross-Platform)

Develop a **lightweight, context-aware client** for caseworkers. Essentially a digital assistant capable of capturing the user's active work environment and relaying that context to an AI backend for assistance.

Key functional goals:

- Detect active applications, document titles, window focus, and relevant metadata via official Windows APIs (and possibly cross-platform alternatives).
- Allow the user to trigger AI assistance via hotkey, capturing context and optional screenshots (user-provided or automated).
- Transmit this contextual information to a backend endpoint (see Part 2) for processing and **receive** AI-generated responses.
- Support a modular architecture enabling future functionality such as task automation or embedded mini-LLMs for local OCR/image processing.
- Include a plugin framework so that new features or integrations can be added independently of the core application.
- Prioritize usability, low system load, and easy deployment to municipal devices.

Technology considerations:

Preferred technologies include C# / .NET with Visual Studio for Windows-native integration, though cross-platform approaches (e.g., Electron, Python + Qt, or Flutter) may also be explored if suitable.

Part 2: Backend / Orchestrator / Integration Architecture

Design and implement a minimal viable **backend service** that serves as the bridge between the client and the municipality's AI infrastructure. This component should orchestrate data exchange with one or more large language models (LLMs) and connected tools through the **Model Context Protocol (MCP)**.

Key objectives:

- Establish a modular backend that can route context data from the client to appropriate AI/ML services
- Explore lightweight orchestration methods that support multiple specialized agents (e.g., retrieval agents for zoning data, document search, or archive lookups)
- Ensure a simple and clear API boundary between the client and backend
- Investigate how open standards and container-based deployment (Docker, FastAPI, etc.) can make the system portable and maintainable across municipal and cloud environments
- Prototype a small-scale MVP that demonstrates end-to-end functionality between the client and backend

Technology considerations:

Python is recommended due to strong ecosystem support for AI, APIs, and orchestration frameworks (LangChain, FastAPI, etc.), but other languages are acceptable. The architecture should avoid dependency on proprietary cloud services; solutions should be open, flexible, and cost-efficient.

Expected Deliverables:

- For Part 1: A functioning context-aware desktop client with plugin-ready architecture.
- For Part 2: A lightweight backend MVP that can receive contextual data and communicate with one or more LLMs through MCP or equivalent mechanisms.
- Documentation covering architectural choices, API interfaces, and data flow.
- Recommendations for scalability, future functionality (e.g., speech, automation), and security considerations.

Figure 2: Proposed project description

Combined, the two parts describe a full-stack AI-powered application aimed at helping caseworkers retrieve relevant information faster and more consistently, without manually searching across multiple systems and resources. The project aligns with a growing movement toward local data processing and independence from proprietary digital services and infrastructure. At a higher level, the project fits within the broader innovation effort of the planning and building department.

1.4 Scope and Limitations

The scope of the project is extensive considering its limited time frame. As this project is likely to lay the groundwork for continued student projects and development in the future, the client does not expect a fully developed end-product. The primary objective is to establish a modular architectural framework connecting the client application with the Municipality's AI infrastructure, with sufficient flexibility to accommodate future changes or additions such as service replacement, specialised agents and workflows, integration of new data sources, or rerouting of requests through alternative endpoints. The solution should also be easy to integrate and adaptable to fit other municipalities and their workflows, underlining the importance of abstraction and portability in the final system.

Additionally, the team defined a specific constraint in regards to platform support. The developed system is built partly with Windows-native software and APIs, meaning the client application currently only targets Windows systems; MacOS and Linux clients are out of scope for this delivery.

1.5 Technologies Used

This project utilises a combination of frontend and backend technologies to support the development of a context-aware AI assistant.

The client application is developed as a Windows desktop application using WinUI and .NET (C#), following the Model-View-ViewModel (MVVM) architectural pattern. The backend is implemented in Python using FastAPI, with Pydantic used for data validation and modelling.

The system integrates with large language models (LLMs) to generate responses based on user input and contextual data. In addition, FastMCP is used to expose tools to the AI through the Model Context Protocol (MCP).

Data is stored in a PostgreSQL database hosted via Supabase, and the database communication is handled asynchronously using asyncpg. Finally, the backend is containerised using Docker.

2. Methodology and Process

The methodology for this project is partly defined by the client and partly by the team. As part of the FoU-2026 program, all teams follow a shared Scaled Agile Framework (SAFe) inspired structure. At project-level, teams are expected to self-organize their working rhythm and tasks. As a result of this, the project follows a combination of SAFe and Scrum methodology principles.

2.1 SAFe and the FoU-2026 Program

The purpose of the shared program structure was to promote direction, synchronisation and delivery across multiple agile teams. As the FoU-program consists of several autonomous student teams, some of whose projects share interdependent functionalities, a shared cadence is necessary to reduce coordination risk and ensure that work can be reused across projects. The cadence is defined through shared working periods called Product Increment (PIs). Within the program, each PI lasts roughly seven weeks, spanning a total of three PIs across the project's duration. In between PIs, teams present what they have produced and what is planned for the next seven-week period. Additionally, regular team syncs, Scrum master forums and architecture forums are held, enabling shared discussion and knowledge sharing across teams within the FoU-program. Rather than implementing full-scale SAFe, the program utilises the parts of SAFe that add value in this setting, primarily PI thinking for shared direction and cadence. This is particularly important as several projects within the FoU-program are interdependent, meaning that progress in one project may rely on progress from others.

2.2 PI Planning

At the start of each product increment, the team prepared a PI plan as part of the FoU-program's shared planning structure. The PI plan functioned as a common planning framework between the team, the client and the wider program. Its purpose was to define the direction for the coming period, clarify expected outcomes, identify uncertainty and make dependencies visible before work began. The PI plan framework used in the project is shown in *Figure 3*.

Team: X

Confidence Vote (commitment)
Team: X
?/10

Primært fokusområde/domene

Relevante Avhengigheter

Pi-mål (Hva forsøker vi å oppnå i denne Pi'en)

Avhengigheter & Risiko (kun det viktigste)

Forventet leveranse (Scope – høyt nivå/birdseye)

Definition of done (DoD - for denne Pi'en)

Kapasitetsantakelse (Hva bygger planen vår på)

Pre-view (neste Pi, etter denne)

Diagram: A diagram on the right side of the form shows a vertical arrow labeled 'Verdientilførsel' (Value Addition) pointing downwards. To the left of the arrow is a circular diagram with icons representing different teams or roles. Below the arrow is a box labeled 'FoU' (Future of Us).

Figure 3: Empty PI plan

Each PI plan consists of several recurring elements:

1. *Primary focus*, describing the overall direction for the product increment.
2. *PI goals*, defining the meaningful capabilities or outcomes the team aimed to achieve.
3. *Expected deliveries*, describing what should be possible to show, explain or demonstrate by the end of the PI.
4. *Definition of Done (DoD)*, clarifying the criteria used to evaluate whether the delivery had sufficient integrity.
5. *Dependencies*, identifying external factors, client input or other teams that could affect delivery.
6. *Risks*, documenting uncertainty that could influence progress.

7. *Capacity assumptions*, estimating the available team effort during the PI.
8. *Confidence vote*, indicating how confident the team was in completing the planned work.

2.3 GitHub Projects

The FoU-program specifies GitHub as the common platform for managing multiple projects simultaneously. It creates a shared platform for keeping track of work and logging progress. Each project has a repository and corresponding GitHub Project used as the primary backlog and roadmap tool. Tasks are organized as issues in a parent-child hierarchy; PI level parent issues contain the PI plan and goals for each product increment, while child issues define the specific tasks required to progress further and meet the defined goals.

2.4 Roles and Responsibilities

Team roles were initially based on DevOps guidelines provided by the program, shown in Figure 4, including Scrum master, release manager, automation architect, security engineer, experience assurance and software developer/tester. DevOps can be understood as a set of practices, tools and a cultural philosophy that aims to iterate and automate processes between software development and IT operations, with an emphasis on collaboration, communication and continuous delivery (Atlassian, n.d.). As part of the FoU-program, each student group is also required to have specific members appointed to attend Scrum master and architecture forums.

Roller i et DevOps team:

Som i et flerfaglig Scrum-team bør utpeking av roller overlates til teamet selv.

Å legge til rette for selvorganiserende og lærende team gjøres best ved å sette sammen riktig kombinasjon av senior- og juniørprofiler med de samlede kompetansene som trengs for å nå målene.

Ethvert smidig team bør ha minst to seniormedlemmer, og ikke et partall totalt – ideelt 5 eller 7 medlemmer.

Et DevOps-team bør ha personer som kan fylle følgende roller:

DevOps-evangelist (Entusiasten)

(Tilsvarende "Scrum Master" i et mer kjent begrepsapparat)

Teamets leder. Fremmer fordelene ved DevOps ved å tydeliggjøre forretningsverdien som oppnås gjennom økt smidighet. Rollen fungerer som en endringsagent som sikrer forankring i organisasjonen. Må kjenne virksomheten, driftsmiljøet og sentrale aktører for å kunne drive adopsjonen fremover. Fokuset ligger særlig på å **legge til rette** for teamet. Rollen bør ha én primær innehaver, men kan rotere mellom teammedlemmer over tid (ikke midt i en sprint).

Release Manager

Ansvarlig for styring og koordinering av produktet fra utvikling til produksjon (tilgjengelig for brukere). Har fokus på de tekniske detaljene i hele leveransekjeden for applikasjonen.

Automatiseringsarkitekt

DevOps handler ikke primært om verktøyene, men verktøyene muliggjør DevOps-praksis. Denne rollen sikrer automatiserte og kontinuerlige utstillinger, samt høy tilgjengelighet i systemene. Automatisering er avgjørende for å opprettholde et stabilt og pålitelig miljø.

Programvareutvikler / tester

Kjernen i DevOps-organisasjonen. Begrepet utvikler/tester brukes for å understreke at utvikleren også har ansvar for testing. Rollen har dermed et bredere ansvar enn i tradisjonell utvikling.

Experience Assurance (XA)

Sikrer at funksjoner som slippes faktisk gir en god brukeropplevelse. Likner klassisk QA, men inkluderer også fokus på selve opplevelsen (f.eks. lastetider, hvor lang tid datagrunnlag tar å oppdatere, etc.).

Sikkerhetsingeniør

Kontinuerlige og automatiserte utstillinger krever at minst én i teamet har kontroll på sikkerhetsmessige konsekvenser. Skal raskt kunne identifisere risiko for informasjonstøkkasje og handle i tråd med interne sikkerhetskrav. Særlig viktig når teamet håndterer sensitiv data.

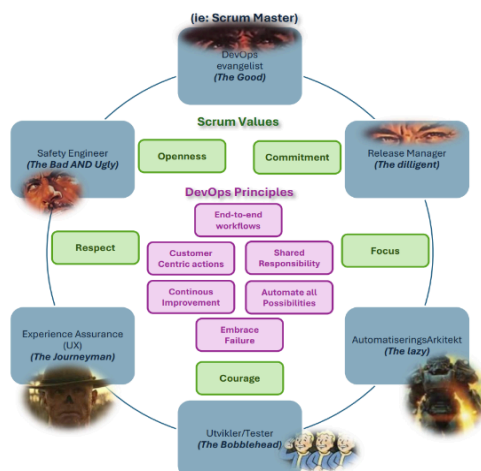


Figure 4: Suggested DevOps roles

These roles were treated as flexible rather than fixed, allowing responsibilities to shift based on project needs. It was also established that all team members share the role of developer and tester as a baseline. While the DevOps roles assign primary responsibility to specific team members, all team members are encouraged to contribute across roles. Figure 5 shows the initial distribution of roles across the team.

Systemansvarlig: Frank

Rolle	Hvem	Ansvar
Scrum Master	Jørgen	Fasilitere møter, fjerne hindringer, vokte prosessen
Release Manager	Frank	DevOps, CI/CD, deployment
Automatiseringsarkitekt	Kata	Testing, automatisering, toolchain
Sikkerhetsingeniør	Jon	Sikkerhet, personvern, datahåndtering
Experience Assurance (UX)	Stine	Brukeropplevelse, testing, demo

Alle er utvikler/tester. Roller kan roteres etter hver Pi (7 uker)

Figure 5: Initial DevOps role assignment

Beyond the DevOps roles, Frank was appointed as program co-admin by the program's tech-lead. The co-admin role includes responsibility for coordinating and facilitating the

shared architecture between student groups in the FoU-program. This includes establishing guidelines and supporting other teams with technical issues related to the shared infrastructure.

2.5 Scrum and Internal processes

The internal, team-level organisation utilises aspects of Scrum methodology, as suggested for the program. Each seven-week PI is divided further into sprints and a concluding PI review week. This approach utilises aspects of traditional Scrum practices, and was decided on internally in the team at the beginning of the project. At the end of each sprint, sprint review meetings are held to discuss achievements, difficulties and potential improvements moving forward. The regular Scrum cadence contributes to more frequent, recurring feedback loops in addition to what the SAFe structure provides.

Stand-up meetings are held every Monday, Wednesday and Friday, where team members cover what they have done, what they will do, and if there are any issues that have occurred. These meetings are carried out digitally over Teams, and are open for external users to listen in on. The weekly office day on Monday prioritises shared discussion and planning, while Tuesdays and Thursdays are kept as individual work days by default. Additionally, the team holds an internal, post stand-up meeting where members handle tasks and redistribute work. This meeting is less rigid than the stand-up and creates space for collective discussion and sharing ideas, though matters that only concern individual members or subgroups are handled outside of it.

2.6 MoSCoW Prioritisation

The MoSCoW method is a prioritisation technique used to categorise different requirements into Must have, Should have, Could have, and Won't Have. This method provides a structured way to manage scope and ensure the optimal prioritisation of development tasks, where time constraints in a project require clear distinction between essential and optional features (Agile Business, 2014).

3. Project Execution

The following sections provide a chronological overview of the project execution, reflecting the Scrum and SAFe structures utilised by the team. Each PI is presented together with its corresponding PI plan, reflecting what the team is committing to the following 7 weeks. For each sprint, key findings, decisions and executed tasks will be presented. The following sprint reflection sections discuss difficulties and improvements regarding internal processes. Each PI section is then concluded with a higher level PI review, including any PI plan deviations.

3.1 Product Increment 1

3.1.1 PI 1 Commit

PI 1 spanned from early January through mid-February, with the team's primary focus being establishing a solid foundation for the system. This included gaining insight into the system's end users; the department's caseworkers, and their current workflows and pain points.

Additionally, PI 1 goals included a core frontend prototype as well as a draft for the system's architecture. Though the scope of the project was difficult to fully grasp at this stage, the team entered the first PI with an average confidence vote of 8/10 as seen in Figure 6.

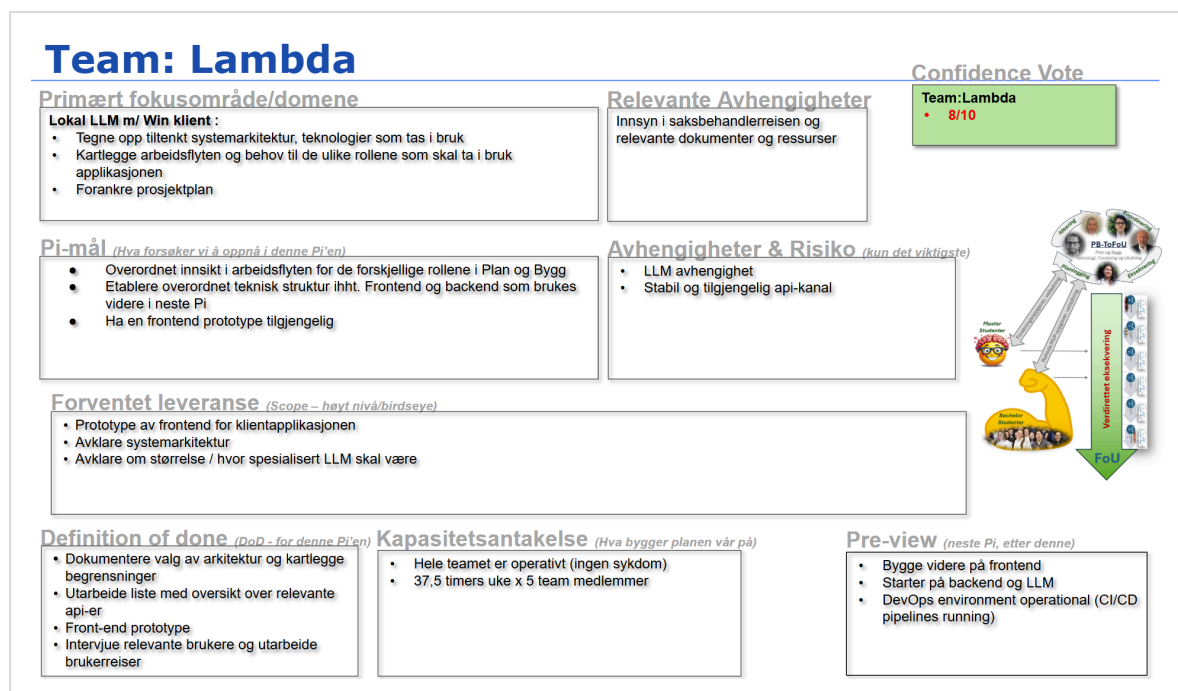


Figure 6: Committed PI 1 plan

3.1.2 Sprint 1 - Analysis and UI Design

The project's first sprint began with understanding and analysing the problem domain, drawing inspiration from the first stages of design thinking process, illustrated in Figure 7, as defined in *Design Thinking på norsk* (Brattli et al., 2023). The stages were worked iteratively, moving back and forth between them as needed.

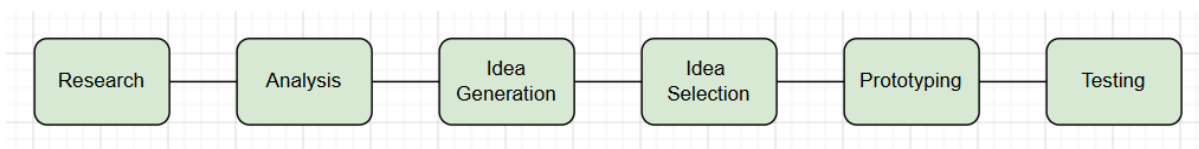


Figure 7: Design thinking process

The team collectively reviewed the project description and additional FoU-program resources to ensure a shared understanding of the project context and its intended direction. To gain a deeper understanding of the system's end-users, semi-structured interviews were planned and conducted with the department's caseworkers. The semi-structured format was intentionally chosen to allow for further questioning in unforeseen topics of interest within the project's scope. The interview questions are included in *Appendix B1-B2: Sprint 1-2 Interview Questions*.

The first interviews gave insight into how caseworkers typically process a case from start to finish. The team focused on identifying relevant inefficiencies and pain points falling within the scope of the developing application. Websites and resources commonly used during a caseworker workflow were also noted. The core workflow processes were finally compiled into a 7-step list of typical activities and pain points. Figure 8 shows a compressed version of the developed workflow. More extensive workflows, as well as other findings from caseworker interviews are included in *Appendix B: Analysis*.

Steg	Aktiviteter & Smertepunkter
1. Åpne søknad	Åpner søknaden i Plan & Build 360 og blar manuelt gjennom tegninger, situasjonsplan og følgebrev, for å forstå hva som søkes om og være forberedt på saken. Noterer selv hva søknaden gjelder. <i>⚠ Manuell gjennomgåelse av mange dokumenter per søknad. Ingen strukturert oversikt: usikker på om alle vedlegg er komplette.</i>
2. Kart & eiendom	For å innhente informasjon om eiendommen benyttes flere separate systemer: kartverktøy (Proffkart, GISLine) for plangrunnlag og matrikelopplysninger, samt

	NVE, Naturbase, Askeladden og DOK-analyse for miljø-, natur- og kulturminneverdier. <i>⚠ Fem til seks systemer for å kartlegge én eiendom: risiko for å overse relevante opplysninger, og ingen samlet oversikt.</i>
3. Plan & lovverk	Gjennomgår ulike planer og lovverk relevant for saken (TEK og PBL) Reguleringsplanen er fra 1987 med utdatert terminologi. Slår opp i Arealplaner.no, DiBK og Lovdata. <i>⚠ Gammel plan-terminologi, må bytte mellom Arealplaner.no, DiBK, Lovdata. Samt tolkningstvil kan stoppe fremdriften. Saken mister en halv dag.</i>
4. Høringer & uttalelser	For å få nødvendige spesialistvurderinger, koordinerer de med interne instanser (ingeniørvesen, parkvesen) og eksterne instanser (Statsforvalter, Vegvesen, Kystverket m.fl.). Gjøres via e-post. <i>⚠ Tidkrevende å koordinere flere instanser, og å følge opp frister.</i>
5. Fatte vedtak	Vedtak føres i Plan & Build og sendes videre til nabo/statsforvalter ved behov. Dersom søknad ikke er i tråd med planen, så vurderes dispensasjon der de søker manuelt i arkivet etter lignende §19-2-saker. Skriver dispensasjonsvurderingen fra bunnen av. <i>⚠ Vanskelig å finne relevante tidligere §19-2 vedtak, utilgjengelige direktivuttalelser. Mangler enkel tilgang til vedtak fra samme planområde/tema.</i>
6. Matrikkelføring	Vedtak registreres i matrikkelen og oversendes til matrikkelfører. <i>⚠ Manuell prosess</i>
7. Gebyrberegning	Gebyr beregnes manuelt, og separat fra matrikkelføring, basert på ressursbruk og sakens innhold. Sendes til fakturaseksjon. <i>⚠ Ingen standardmål: utsatt for feil og inkonsistens på tvers av saker.</i>

Figure 8: Planning and building caseworker workflow and pain points

Lastly, caseworkers were asked about their current usage of AI tools within their daily workflows. The answers uncovered familiarity with tools such as OpenAI's ChatGPT and Microsoft's Co-Pilot, though their usage within daily workflows would vary. The first rounds of interviews provided the groundwork for more detailed user journeys and personas as seen in *Appendix B: Analysis*.

With a clearer picture of the problem domain, focus shifted towards application design. Interview findings, alongside the project description and common UX design principles guided the design process. Jakob's Law states that “users prefer your site to work the same way as all the other sites they already know” (Yablonski, n.d). This was established as a central design consideration to reduce the learning curve following the adoption of a new system or tool. With the interviewed caseworkers having existing mental models for major AI-tools, the team deliberately drew inspiration from applications such as Claude and ChatGPT in the UI design process.

Application sketches were developed individually and discussed collectively, gathering the strongest ideas. A key structural decision emerging from the 7-step workflow was the separation of conversations into cases; grouping related chats, documents and resources under a shared context. This resembles the grouping of conversations into ‘projects’ in ChatGPT or Claude, intended to provide structure as caseworker interviews identified lack of organisation as a core problem in their workflows. Initial application pages were developed into a navigation map as seen in Figure 9, consisting of the core views the user would be able to navigate through.

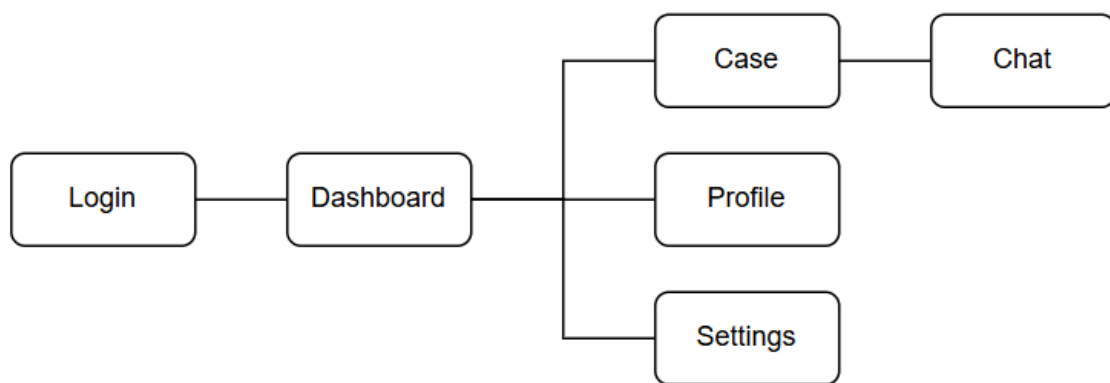


Figure 9: Initial navigation map

Based on the sketches and design discussions, corresponding wireframes were developed. The wireframe for the ‘CasePage’ view can be seen below in Figure 10. For a complete overview of the initial design process products, see *Appendix C: Design Material*.

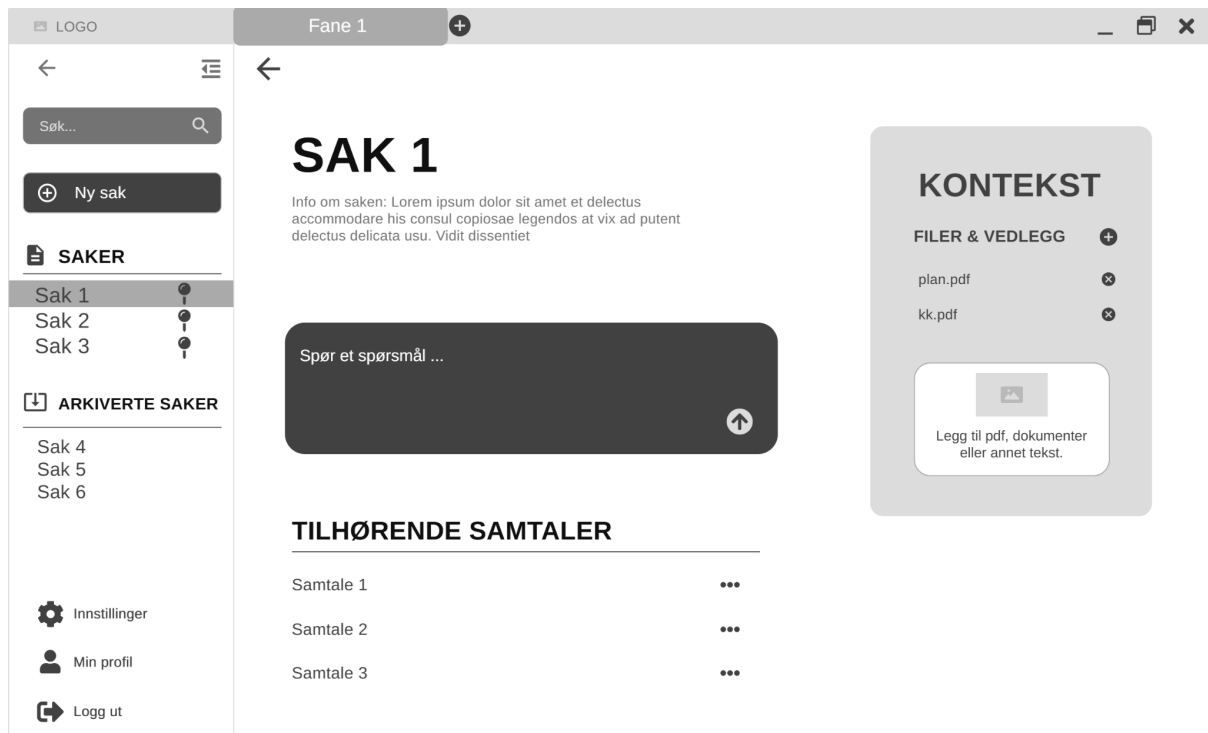


Figure 10: CasePage wireframe

As the design took shape, a frontend framework needed to be chosen. This marked the *first major architectural decision* in the project. The project description, shown in Figure 2, defines two alternatives for the frontend application; either a Windows-native or cross-platform approach. After collective discussion and research, the team concluded on utilizing a Windows-native approach given the following considerations:

1. The project description specifies a preference for using .NET and C# in Visual Studio, supporting a Windows-native approach.
2. The team members have prior experience with .NET and C# in Visual Studio.
3. Utilising official Windows APIs for capturing context is specified as a key functional goal for the context-aware client application, which may favour using a Windows-native framework.
4. Based on caseworker interviews, the majority, if not all caseworkers, utilize Windows computers distributed by Kristiansand Municipality.

After further research on Windows-native frameworks, the alternatives were narrowed down to Windows Presentation Form (WPF) and WinUI3. Though WPF is the more mature option of the two, the team landed on WinUI as the final frontend framework. WinUI offered more modern UI design capabilities, as well as official tools such as the WinUI Gallery and

Toolkit. The tools include visual examples of UI components, in addition to corresponding example code. This was important for the team to to reduce the learning curve of adopting a new frontend framework.

As a direct consequence of choosing WinUI 3, the Model-View-ViewModel (MVVM) architectural pattern was adopted over the more familiar Model-View-Controller (MVC) pattern. Based on the team's research, MVVM aligns more naturally with WinUI's intended separation of concerns (Windows, 2024), facilitating a more maintainable and scalable frontend architecture. Similarly to the previously mentioned WinUI tools, the availability of an official MVVM toolkit strengthened the team's confidence in utilising the intended WinUI architectural pattern in this project.

3.1.3 Sprint 2 - Frontend Prototype and System Architecture Design

With sufficient groundwork in place the focus shifted more towards implementation. The frontend WinUI application was created. Predefined WinUI components were identified and mapped accordingly to the previously defined wireframes. Lastly, the wireframes were implemented as views in the WinUI application, as illustrated in *Appendix D1-D6*. As mentioned earlier, the design thinking stages were worked iteratively, meaning UI design updates were continuously discussed and integrated as the team developed a deeper understanding of caseworkers and their needs. The first iteration of the 'CasePage' view can be seen below in Figure 11.

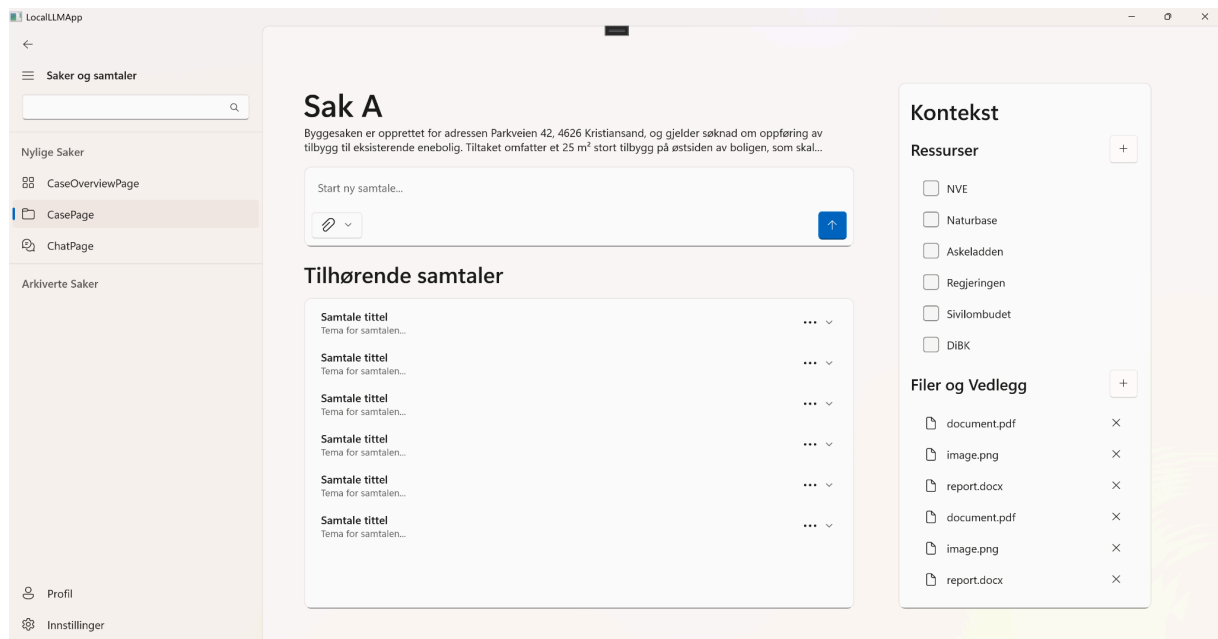


Figure 11: CasePage view

Parallel to the frontend development, further reading on the MVVM pattern and its implementation was carried out. Continued research on WinUI was conducted, enabling development of specialized and reusable UI components for the application. To support these processes, the team utilised WinUI 3 Gallery, MVVM Toolkit and Windows Community Toolkit to inform implementation choices and overall development.

Additional user interviews were conducted with relevant parties within Kristiansand Municipality, providing deeper insight into the department's archive documents. Caseworkers specialized within supervision & illegality cases were also interviewed. Their corresponding workflow and pain points are included in *Appendix B3: Supervision & Illegality Caseworker workflow and pain points*. The new interviews showed that many of the previously identified pain points significantly overlap within the different caseworker roles of the department. They utilize the same systems, and experience time-consuming and fragmented information retrieval across sources.

The team continued to iterate on the system design. The project repository was restructured into two main folders, separating concerns between the frontend application and backend server. Early architectural sketches were refined, and internal discussions on data structures and API designs were held, though with varying success. After including the program's tech-lead in the discussion, the team was suggested an initial approach which mocked APIs.

This method would enable the team to simulate aspects of backend behaviour. However, progress on this front was limited, as the team's understanding of the backend architecture and technologies was not sufficiently established. Due to this gap, mock API implementation was postponed.

Towards the end of the sprint, the team shifted focus towards establishing a foundational understanding of the backend architecture. This was driven in part by earlier challenges related to mock API implementation and uncertainty surrounding API design. As a result, the team conducted research into central backend technologies such as FastAPI, Pydantic, Docker and Model Context Protocol (MCP), with the aim of identifying suitable tools and architectural patterns for the system.

3.1.4 Sprint 1 Reflection

In the beginning of the first sprint, the team lacked sufficient knowledge on the required technologies and the department's caseworker processes. Additionally, internal processes and rhythm were not settled into. Difficulties with scoping and planning the road ahead were brought up, as completed tasks often lacked a clearly defined next step. Further discussion highlighted potential improvements to internal processes, regarding the logging of work hours, discussions and tasks. These logs are included in *Appendix F: Logging of hours*. Routine hour logging had been lacking the entirety of the first sprint, and discussion and issues had no standardised structure. Without shared and structured logging, the team lacked a reliable source of reference for future discussions and improvements.

As a result of the first sprint review, a standardised log structure was introduced. Structured documents for capturing working hours, as well as a daily log summarizing completed work, planned tasks, and team discussions were created. This led to several positive outcomes: the team could more easily track progress, identify bottlenecks early, and ensure transparency in work allocation. Additionally, the structured logs lead to smoother handovers when team members were absent, and made retrospectives more data-driven. Following these improvements, structured logging was established as a routine across the team. As for the difficulties in planning, the team managed to set the project in motion through collective brainstorming and discussion. This greatly aided the team in gaining a deeper understanding

of the project and problem domain, and was noted as one of the major successes of the first sprint.

3.1.5 Sprint 2 Reflection

During the second sprint review, two major difficulties were discussed. First, the team had been struggling to break down PI plan commits into smaller, more manageable tasks. Shared discussion identified a gap between the lowest level issues and highest level PI goals. Additionally, pinpointing and estimating progress according to the PI plan was difficult, as the team had not divided clear goals for each sprint. Secondly, the team struggled with meetings running longer than intended, reducing effectiveness and focus on individual tasks. Though longer meetings and collective discussion had greatly benefitted the team in the starting phases of the project, tasks had grown to be increasingly specialised during the second sprint. Long and unstructured meetings started interfering with individual work time, disrupting the focus required to make progress on distinct areas and technologies within the system.

As a fix for managing tasks and progress, the team agreed to further divide the high-level PI goals into mid-level sprint goals. By breaking down the large goals into smaller, more manageable goals, it is easier to divide work between the two sprints in each product increment. Additionally, planning and defining issues at the beginning of each sprint becomes simpler. To address the prolonged meetings and discussions, the team agreed on a clearer weekly structure going forward. Mondays would serve as planning and discussion days, while Tuesdays and Thursdays would be kept as individual work days. All internal, post-stand-up meetings would open with a fixed agenda or backlog review, ensuring discussions stay on topic and that meetings are both efficient and relevant for all members.

3.1.6 PI 1 Review

Though the first PI plan was committed prior to gaining more detailed insight of the problem domain, the scope of the goals and expected deliveries turned out to fit well for the starting period. At the end of the first PI, the team had acquired a solid understanding of the department's caseworkers. Caseworker processes and pain points had been mapped, contributing to a stronger understanding of the problem domain. As for system development,

the core UI design for the client application was implemented and an initial plan for the system's architecture was established.

At the end of the first PI, three points deviated from the initially committed PI plan, shown in Figure 12. The description of LLM size and 'specialisation' was omitted from the product increment delivery. As the team developed a better understanding of the system, work regarding LLMs and their integration were deemed better addressed at a later stage in the project. As for the two partially finished Definition of Done points, mapping relevant APIs and architectural limitations would become iterative tasks continued as the project progresses.

Team: Lambda

Primært fokusområde/domene

Lokal LLM m/ Win klient :

- Tegne opp tiltenkt systemarkitektur, teknologier som tas i bruk
- Kartlegge arbeidsflyten og behov til de ulike rollene som skal ta i bruk applikasjonen
- Forankre prosjektplan

Relevante Avhengigheter

Innsyn i saksbehandlerreisen og relevante dokumenter og ressurser

Confidence Vote

Team: Lambda
• 8/10

Pi-mål (Hva forsøker vi å oppnå i denne Pi'en)

- Overordnet innsikt i arbeidsflyten for de forskjellige rollene i Plan og Bygg
- Etablere overordnet teknisk struktur ihht. Frontend og backend som brukes videre i neste Pi
- Ha en frontend prototype tilgjengelig

Avhengigheter & Risiko (kun det viktigste)

- LLM avhengighet
- Stabil og tilgjengelig api-kanal

Forventet leveranse (Scope – høyt nivå/birdseye)

- Prototype av frontend for klientapplikasjonen
- Avklare systemarkitektur
- Avklare om størrelse / hvor spesialisert LLM skal være

Definition of done (DoD - for denne Pi'en)

- Dokumentere valg av arkitektur og kartlegge begrensninger
- Utarbeide liste med oversikt over relevante api-er
- (Visuell) Front-end prototype
- Intervjue relevante brukere og utarbeide brukerreiser

Kapasitetsantakelse (Hva bygger planen vår på)

- Hele teamet er operativt (ingen sykdom)
- 37,5 timers uke x 5 team medlemmer

Pre-view (neste Pi, etter denne)

- Bygge videre på frontend
- Starter på backend og LLM
- DevOps environment operational (CI/CD pipelines running)

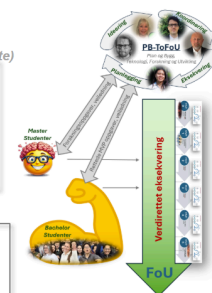


Figure 12: End of PI 1 plan

3.2 Product Increment 2

3.2.1 PI 2 Commit

PI 2 spanned from late February to mid April. Building on the foundation established in PI 1, the team committed to delivering a full-stack prototype with end-to-end data flow across all the internal system endpoints. A secondary focus was continued iteration on the frontend. The overall goal was to enable a user to submit a text query to the LLM through the frontend, and receive a contextually relevant response. The entire plan for PI 2 is shown in Figure 13.

Given the scope and due to the team being unfamiliar with several of the involved technologies, the confidence vote averaged at 7/10 at the start of PI 2.

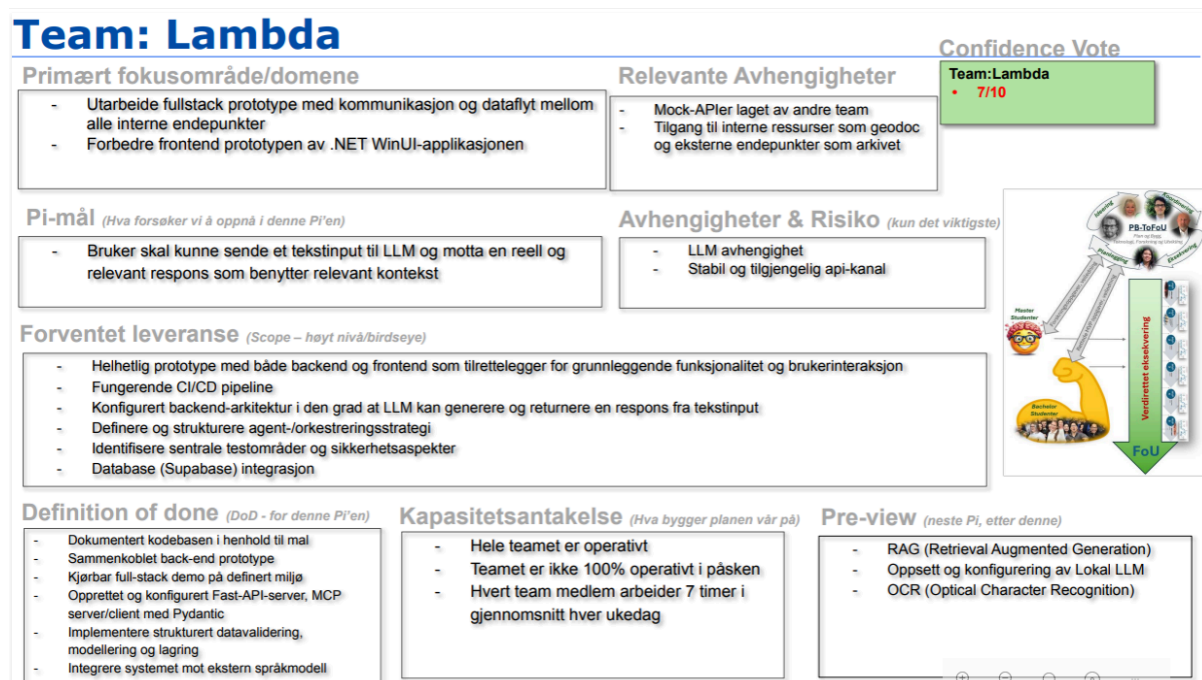


Figure 13: PI 2 plan

3.2.2 Sprint 1 - Establishing the Backend Foundation

The primary focus of Sprint 1 was to establish a robust backend foundation and ensure end-to-end connectivity between the frontend and backend of the application. Initial work centered on introducing core backend technologies, including FastAPI and Pydantic, and validating system communication through an early integration between the LLM and an MCP-based server/client. Work on database integration began at the end of Sprint 1, addressing a longstanding backlog issue, and continued into sprint 2. Although substantial progress on the database was limited before the end of PI 2, these efforts laid the foundation for a more comprehensive data management integration into the following sprints. This implementation confirmed that the core components could communicate correctly, supporting the feasibility of the architecture in the initial design and implementation phase.

Development then focused on defining structured data models in Pydantic based on relevant municipal data sources, as well as research of AI agents and their implementation.

In parallel, some of the core frontend-backend communication was successfully implemented, demonstrated through functional chat integrations. This provided a working pipeline for user input and a system response.

On the frontend and UX side, a meeting with a caseworker was conducted to evaluate the user journey and UI, providing feedback that informed priorities moving forward. The session clarified how caseworkers envision using the system in practice: the LLM should help facilitate structured reasoning and support the retrieval of relevant prior decisions, and supports consistent language and argumentation by reinforcing their judgment rather than replacing it. Another finding from the conversation was that the terminology used in early wireframes (e.g., "Kontekst") was nonintuitive for caseworkers, leading to adjustments in labeling and information architecture, described further in *Appendix B9: Caseworker Feedback on User Journey*.

A MoSCoW-based development plan was created for the frontend to align the team on implementation priorities, described in *Appendix B10: MoSCoW Analysis Frontend*. The plan included which components could be developed using data bindings with placeholder data, enabling frontend work to begin while backend integration was still being defined. Additionally, further exploration of relevant Windows APIs was undertaken to support future context gathering.

3.2.3 Sprint 2 - Backend Architecture and Scope Adjustments

Sprint 2 opened with uncertainty around how to proceed with testing the endpoints against real data. A discussion with the program's tech-lead clarified priorities and led to a key architectural decision. Rather than attempting to integrate all relevant municipal data endpoints, the focus should be on building a backend capable of mounting and dismounting any endpoint dynamically. This implied prioritizing flexibility and scalability over short-term completeness. A visualisation of this is included under *Appendix E6: Endpoint Architecture*.

As a result, the team shifted focus from targeting specific caseworker workflows to designing a more abstract and scalable backend foundation. This shift was influenced by increasing complexity of the system, limited familiarity with the technology and the need to stay

adaptable within the project's time constraints. The implications of this decision is discussed further under section 3.2.6 - *PI 2 Review*.

To better map the system's data flow, a rough sequence diagram was produced to visualize how the input travels through the system and back to the client application, illustrated in *Appendix E7: Sequence Diagram*. Simultaneously, research on orchestration was conducted. The application lacked a sophisticated orchestration layer; the logic that determines how an incoming client request is processed into a useful response. At this stage, the user could communicate directly with an LLM, though the responses had no tools or context other than that which the user had textually provided. A range of sources could aid in producing more informed and useful responses in the system's backend, including the database, external resources and APIs, MCP tools, and usage of different LLM models. The orchestration layer would decide which of these are relevant for a given request.

Two alternative approaches were identified for orchestration; a rule-based routing system, and agentic orchestration. After further research into the different alternatives and a plethora of frameworks, the team concluded on an agentic orchestration layer using Pydantic AI. The main reason supporting this decision lies in that it allows for flexible handling of ambiguous requests, with possibilities for adding more constraints if needed. Pydantic AI also integrates well with the current architecture using FastAPI and Pydantic, and supports further configuration of specialized agents in the future.

As a direct result of this choice, previous MCP solutions were replaced with a simple FastMCP server mounted on the current FastAPI server. The FastMCP server would hold tools readily available for the Pydantic AI agents to use. These changes lay the foundation for producing better responses to the user and orchestrating workflows in the backend. Figure 14 shows the integration of Pydantic AI and FastMCP into the system's workflow.

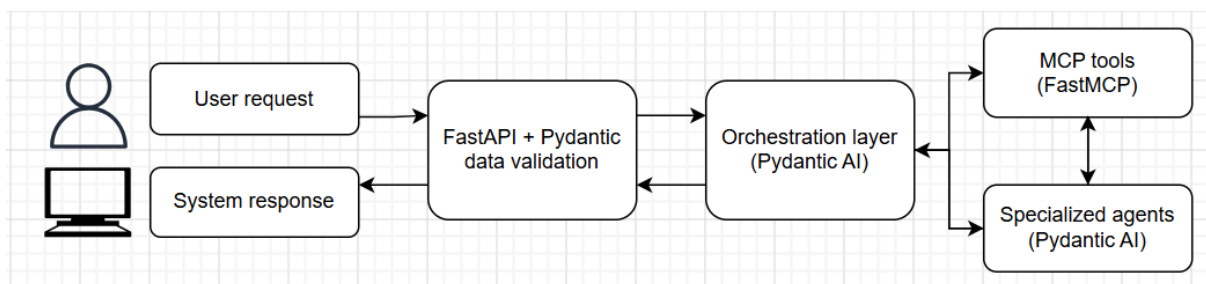


Figure 14: *Orchestration and agentic system workflow*

As part of the backend development, an initial database structure was established to support storage and retrieval of application data. The structure reflects the systems core entities: user, case, chat and chat messages. An overview of the database structure is shown in Figure 15.

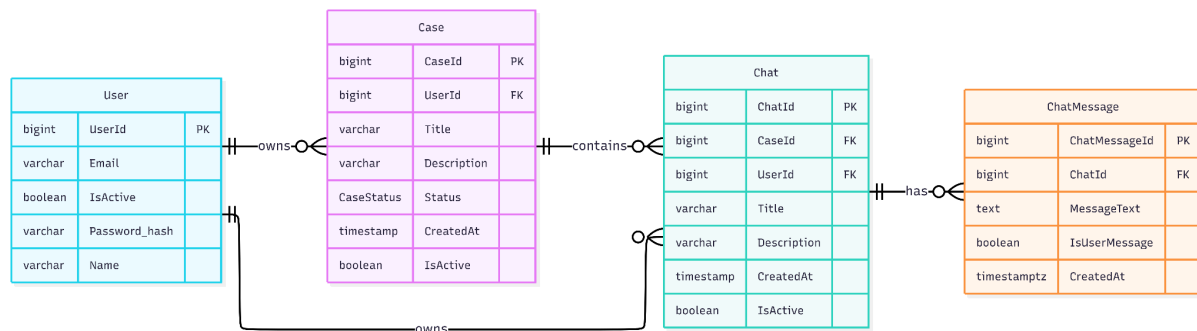


Figure 15: Early database schemas

On the frontend, UX fixes were applied to chat components, automatic test credentials for login were implemented, and chat and case creation were made functional. These improvements moved the prototype closer to a realistic user workflow.

With the architectural direction clarified and core backend structures in place, the team used the transition into review week to connect the layers of the system. The frontend components were mapped to identify where data binding was required, and corresponding data models were implemented. This enabled the database to handle incoming data more effectively, while also allowing UI components to behave more dynamically based on the underlying data structure.

3.2.4 Sprint 1 Reflection

At the conclusion of Sprint 1, the team identified challenges related to task breakdown and delegation, largely due to limited familiarity with newly introduced backend technologies. Many backlog items required additional research before they could be properly estimated, which impacted planning accuracy.

To address this, the team introduced pairing on related tasks to reduce the risk of pursuing unproductive approaches independently. In addition, cleared communication practices were encouraged, including formulating questions in advance and being more transparent on progress. This improved alignment within the team and enabled earlier identification of issues.

The structure of backlog items was also reviewed, identifying the need for more detailed and consistent issue descriptions following a template the team created. Improving this structure was seen as necessary in order to ensure continuity and allow tasks to be more easily shared or reassigned when needed.

3.2.5 Sprint 2 Reflection

During Sprint 2, the team demonstrated improved structure in both backlog management and daily workflow. The introduction of standardized issue templates led to more precise task descriptions, clearer scope and better distribution of work. This reduced ambiguity when picking up tasks and made it easier for team members to contribute across areas when necessary.

Meetings became more focused and efficient. Agendas were introduced for post stand-up discussions, which helped maintain that conversations were kept relevant to the entire team. Topics concerning specific components were handled in subgroups. This reduced unnecessary context switching and allowed the other non-concerning team members to spend more time on development.

Despite these improvements, estimation of the tasks remained a challenge. Work involving new technologies, particularly within backend development, often required additional research before the implementation could begin. As a result, the team identified re-estimation as an ongoing activity, rather than a one-time step during planning.

Gaps between stand-up meetings also contributed to reduced visibility into ongoing work. In several cases, obstacles were not identified fast enough. To address this, the team proposed introducing informal check-ins on Tuesday and Thursdays in order to more efficiently identify obstacles.

Communication improved overall, but some inconsistencies remained. Issues were sometimes brought up too late or communicated too broadly, for example through general messages rather than directly to the relevant team members. This delayed the process of solving problems. To improve this, the team agreed to communicate issues earlier and more

directly to those it concerned. In addition, formulate questions in advance which enables for more efficient discussions.

Looking ahead to PI 3, the team identified a need for more iterative planning. Rather than fully defining sprint tasks upfront, an initial high-level planning which is then later refined after the necessary research has been conducted. This would allow for more realistic scoping as the team's understanding of problems improves.

Finally, Frank's role as Co-Admin across all bachelor teams introduced an additional constraint on the teams capacity, as it required coordination and support beyond the immediate scope of the project. While this limited the internal capacity, it also contributed to the development across the other teams and supported the establishment of a shared architectural foundation.

3.2.6 PI 2 Review

PI 2 introduced several new technologies to the team, and the initial scope slightly changed through this period. Despite these changes, the originally committed objectives remained aligned with the intended outcomes and deliverables. Significant effort was dedicated to conducting research and implementing new technologies on the backend, with particular focus on establishing frontend to backend connection.

At the end of the PI, the team had completed nearly all planned objectives, as shown in Figure 16. The primary focus of a full-stack prototype was marked as partially completed, due to a partially finished database flow. Once completed, the remaining partially completed items were expected to be finalised as a result. The only exception is code documentation, as parts of the codebase were refactored and have not yet been fully documented in accordance with the team's template. The documentation will be addressed in the next PI, ensuring a proper product of quality for the final delivery.

Team: Lambda



Figure 16: End of PI 2 plan

A defining decision in PI 2 was the architectural shift towards a more flexible and abstracted backend. This shift led to a deliberate trade-off. While it limited immediate user functionality, it increased the long term value of the system. The resulting architecture enables future integration to data sources, tools and other services without requiring a fundamental redesign. In this sense, the team prioritized building a scalable and reusable foundation over delivering a narrowly scoped prototype.

At the same time, certain complex areas were consciously deprioritized. In particular, implementing relevant context and OS-level context capture remained incomplete. These areas were identified as highly complex and time-intensive, with the potential to significantly impact the project scope. Based on architectural guidance and time constraints, the team decided not to fully address these challenges during PI 2, but rather favour core system functionality and flexibility.

Similarly, aspects related to security and privacy were not fully implemented during this phase. These were acknowledged as important functionalities, but were intentionally scoped out in favour of implementing core system functionality and architectural flexibility. Known limitations in these areas are not accidental, but a result of conscious prioritization.

3.3 Product Increment 3

3.3.1 PI 3 Commit

For the final product increment, the team defined a PI 3 plan, illustrated in Figure 17. Given the limited time left and a vast scope of work to choose from, the team initially brainstormed tasks and filtered them into a MoSCoW-priority list. The PI 3 plan is composed of the ‘must-haves’ from this list, ensuring the last period focuses on the most important work.

The main goal for this commit was to deploy an MVP which can capture context, upload documents and receive AI-assistance through a local LLM instead of an external LLM. Implementation of the GeoDoc API and local LLM were identified as direct dependencies to the client, as the team relies on Kristiansand Municipality to be able to incorporate the corresponding functionality.

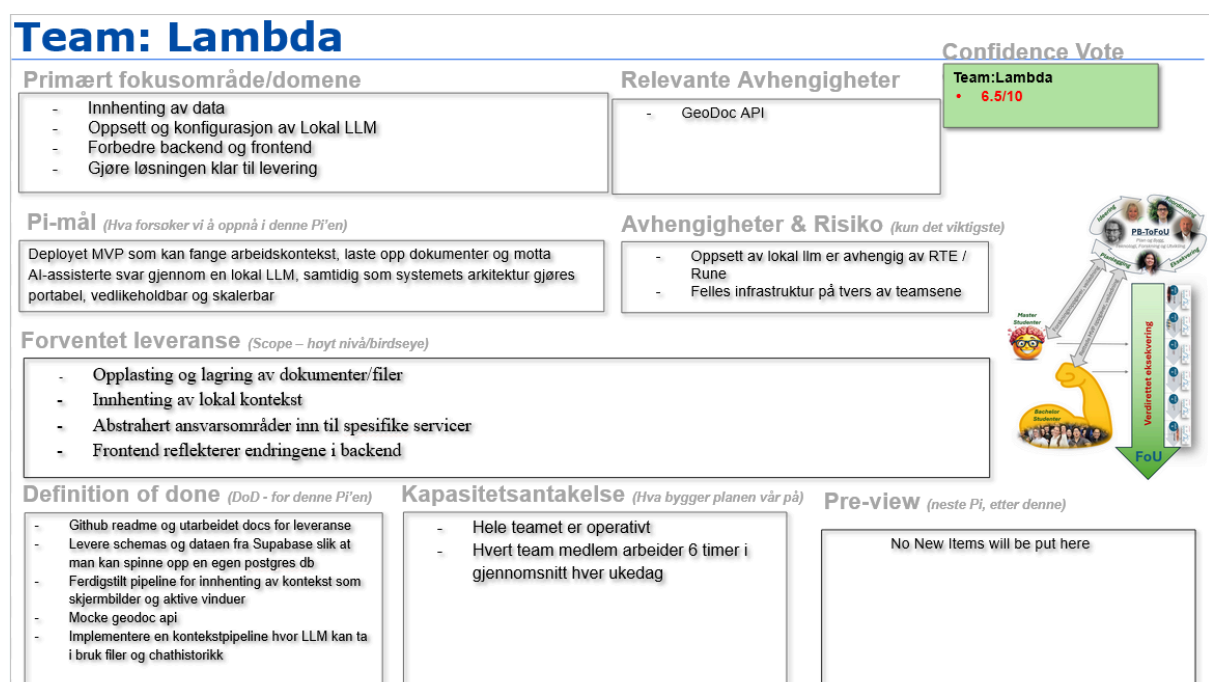


Figure 17: PI 3 plan

Due to this PI being cut short, the team decided to reduce sprint duration from 3 weeks to 2 weeks. The team reasoned that shorter sprints would split the committed tasks into more manageable and focused working periods, taking into consideration other events such as

presentations and exam dates. Additionally, this enables the team to more accurately pinpoint work progression and re-estimation up to the planned sprint goals.

3.3.2 Sprint 1 - Final Prioritisation and MVP Development

Sprint 1 of PI 3 marked the beginning of the final phase of the project. Due to the limited timeframe of this PI, tasks were more strictly prioritised and delegated to ensure an efficient division of labour. The team split into subgroups, where some members focused on development while others worked on the report. Daily status meetings were used to maintain a shared understanding across the different workflows.

Development work during the sprint included improvements to database functionality, including case deletion, archiving and reactivation. Authentication issues were addressed, and file upload and storage functionality was implemented. These tasks were prioritised because they contributed directly to the minimum viable product and supported the intended user workflow. As part of these changes, the underlying database schemas were updated to reflect the expanded functionality on file handling. The revised schema is illustrated in *Figure 18*.

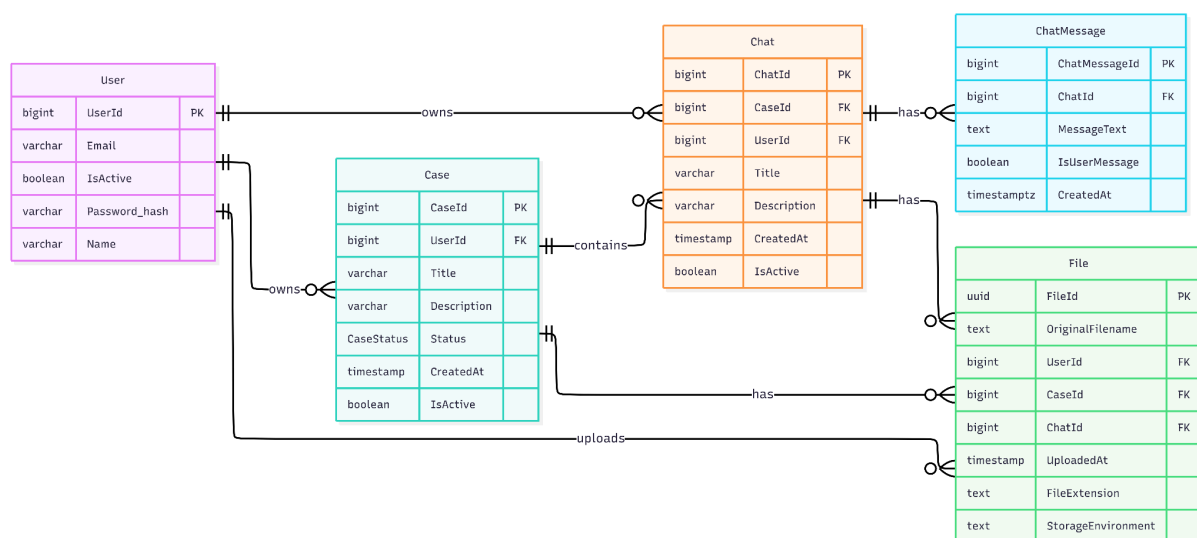


Figure 18: Current database schemas

3.3.3 Sprint 1 Review

During sprint 1, the team demonstrated an improved ability to reestimate tasks and communicate available capacity continuously. This contributed to a more realistic planning process and allowed the team to better track progress towards sprint goals. Dividing work

into subgroups, combined with more frequent standups, improved team coordination and enabled a more efficient allocation of both development and report work.

Despite the initial prioritization and estimation of tasks, the sprint was affected by unexpected circumstances, limiting the team's overall available capacity. With a tightly scheduled plan and reduced timeframe, there was little leeway for delays or unexpected challenges. This was handled to a best possible extent through continual reestimation of work and capacity, as well as iterative prioritization of tasks.

3.3.4 Remaining Work

Following Sprint 1, the remaining work primarily concerns completion of the context pipeline, as well as further backend and frontend refinement. On the backend, this includes continued work enabling the LLM to use relevant context, as well as further abstraction of data handling. On the frontend, remaining work involves improving navigation and adding functionality to make the prototype more usable in realistic caseworker workflows.

From a UX perspective, planned user tests intend to validate core workflows and recently implemented functionality. The first round of testing intends to focus on whether caseworkers could understand and navigate the application, while a later test concentrates on extracting broader feedback on usability and overall system value. Insights from these tests will either inform immediate refinements or be documented as recommendations for future development, depending on implementation complexity and remaining time.

In addition to the work based on the PI 3 plan, the team identified additional future functionality which would most likely not be addressed during the timeframe of the bachelor's project. These functionalities are elaborated on further in section *4.3 Future development*.

4. The Final Product

4.1 Application Demo

The application demo shows aspects of a typical caseworker workflow, and reflects the current state of development as of PI 3, sprint 2. Corresponding images of the WinUI application are included in *Appendix D7-D14*. The demo video is uploaded to YouTube:

<https://www.youtube.com/watch?v=bOFWErXhd3M>

4.2 System Architecture

As shown in Figure 19, the system is designed as a client-server architecture which consists of three primary layers: a desktop client, a backend service and an external AI service.

The frontend layer uses a WinUI3 desktop application which is based on .NET. It handles user interactions and captures relevant context, such as active windows and metadata, which is transmitted to the backend.

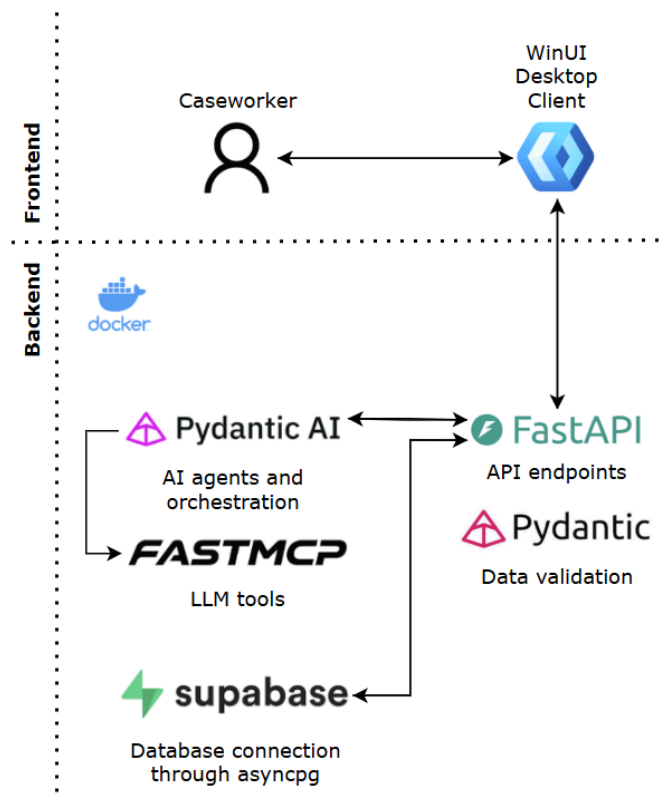


Figure 19: System architecture

The backend is implemented using FastAPI and deployed in a Docker container on a virtual machine (VM). It serves as a centralized layer managing incoming requests from the client, coordinating communication to the different backend endpoints.

Pydantic is used to define structured data models for request and response handling. These models act as a contract for objects such as login data, chat messages and users, ensuring validation of incoming data before backend logic is executed.

The backend interacts with a PostgreSQL database hosted via Supabase. To align with the LLM integration, `asyncpg` is used to perform asynchronous operations to prevent the database queries from getting blocked during requests. `Asyncpg` was selected over other alternatives such as `psycopg` and full Object Relational Mapping (ORM) frameworks like SQLAlchemy, as it is lightweight, provides high performance and has a low barrier for executing raw SQL.

The orchestration layer is implemented with Pydantic AI. An orchestration agent is called through a FastAPI endpoint, which generates an appropriate response based on the given request. In addition, the backend incorporates FastMCP, which binds FastAPI and the Model Context Protocol, providing the agent with external tools. To manage incoming traffic, Traefik is used as an entry point on the VM, exposing backend API endpoints and forwarding requests.

The chosen architecture enables flexibility and scalability through an abstracted and modular backend that supports dynamic mounting and dismounting endpoints. In addition, abstraction and mock based development allows for integration of external dependencies without reliance on real data sources. This allows external data sources and services to be integrated without major structural changes. The separation between frontend and backend supports loose coupling, allowing components to be integrated independently while reducing complexity and risk while adding new functionalities.

As a result, the architecture provides a scalable foundation for further development, where new capabilities and system integrations can be added without a fundamental redesign. This reflects the team's prioritization of long-term adaptability over short-term results.

4.3 Future Development

Due to constraints within the scope, time and cost, and the fact that the system has been developed entirely from scratch, several aspects of the application could not be fully implemented within the timeframe of the project. As a result, potential features have been identified using the MoSCoW method. While these features are not part of the current implementation, the system architecture has been designed to support their integration without requiring major structural changes.

4.4.1 Retrieved-Augmented Generation (RAG)

A Retrieval-Augmented Generation based context pipeline is a central future component. RAG optimises the output of LLMs and references knowledge outside of its training data before generating a response, making the knowledge up to date and thus reducing hallucinations (Amazon, 2023).

4.4.2 Direct Archive Integration

Integration with the municipalities archive will provide direct access to documents through an API endpoint. Case workers spend a lot of time manually searching for documents. As a result, this integration will help reduce manual document handling and streamline workload of case workers,

4.4.3 Optical Character Recognition (OCR)

Optical Character Recognition will enable extraction of text from images and scanned documents. By implementing this feature, the system will significantly broaden the range of compatible data sources. A majority of the documents in the archive are handwritten or scanned. Without OCR, this information would remain inaccessible for the context pipeline (Amazon, 2022).

4.4.4 Web Tokens for Session Management

The implementation of web tokens for session handling will provide a more secure authentication and authorization layer.

4.4.5 Specialized Agents and Workflow Orchestration

Future development may include the introduction of specialized agents workflow orchestration to handle complex tasks more efficiently. Instead of relying on a single agent to handle tasks, the system can utilize multiple agents, each responsible for a specific function such as document retrievals, summarizations or context-enrichment. This design is beneficial when integrating features like RAG and OCR, as it allows the system to generate responses through multiple steps rather than relying on one single process.

4.4.6 Conversation Compression

To address context window limitations in LLM-based systems, future development may include conversation compression techniques. This involves automatically summarising or compressing older parts of a conversation while preserving the most recent interactions, thus enabling longer sessions without exceeding context limits. Such an approach ensures that relevant information is retained while reducing the token usage and improving performance.

4.4.7 Structured Logging Tools

Observability tools such as Pydantic Logfire could be implemented to monitor the system behaviour, debug agent workflows and analyse requests during runtime. This would provide improved visibility into how the system operates in production, making it easier to trace issues and bottlenecks.

5. Reflection

The following sections provide reflections on quality assurance and resource management, as well as a retrospective discussion of what the team would do differently in hindsight. Lastly, the major learning outcomes of this project are reflected upon. Further individual reflections on roles and contributions, as well as team-level reflection is included in *Appendix G: Self and Team Evaluation*. Additionally, the client's final statement on the project is included in *Appendix H: The Client's Evaluation*.

5.1 Quality Assurance and Resource Management

The project management triangle, shown in Figure 20, balances the constraints of scope, time and cost, and has framed the team's quality-assuring decisions throughout the project. As quality is subjective and context-based (Microsoft, n.d), it is the team's responsibility to adjust the constraints to ensure quality in deliveries.

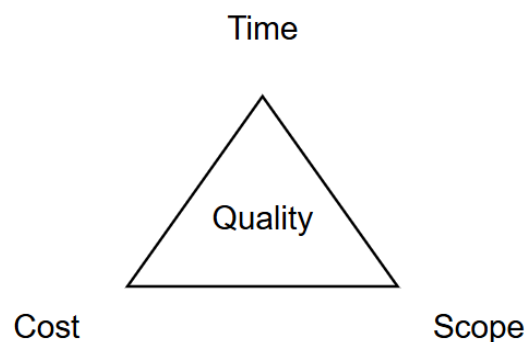


Figure 20: Project Management Triangle

Time reflects the duration of the semester, limiting the team's development to a few months. Cost refers to the allocation of resources. In the context of this project it primarily represents the team's capacity. Finally, scope defines the focus of the project. Some aspects of these three constraints are fixed, such as the PI cadence and final deadline for project delivery, the fixed maximum number of team members, and the core ideas driving the system development. Despite this, other aspects of time, cost and scope are adjustable, enabling the team to balance constraints according to the project context.

Internal deadlines were adjusted to accommodate changing priorities and emerging challenges, while the scope was continuously refined based on user interviews, technical discussions and client feedback. Cost, understood as team capacity, proved to be particularly important because research, implementation, coordination and process improvement all drew from the same limited pool of effort. Quality assurance was therefore closely connected to how the team planned, prioritised and redistributed available capacity throughout the project.

A recurring challenge was that early planning assumed more effective capacity than was realistically available. The initial PI plans did not fully account for coordination overhead, technical dependencies, research activities or unexpected complexity caused by unfamiliar technologies. As the project progressed, research time was increasingly treated as a separate work activity rather than being included implicitly within implementation estimates. This made planning more realistic and improved the team's ability to assess whether tasks could be completed within the available timeframe.

Prioritisation became central to maintaining quality within these constraints. The committed PI plans helped narrow the scope into manageable delivery periods, while the MoSCoW method was used to distinguish essential functionality from less critical features. This allowed the team to focus on core functionality while postponing features that were valuable but not necessary for the final prototype. Continuous reestimation also became important, as tasks involving unfamiliar technologies often proved more complex than initially expected.

Another important resource consideration was the balance between specialisation and shared understanding. Dividing work according to primary areas of responsibility reduced unnecessary context switching and helped utilise limited capacity more effectively. At the same time, this created a risk of knowledge silos, especially when frontend and backend work depended on shared data structures and interfaces. To reduce this risk, the team increasingly relied on written summaries, subgroup discussions and collaborative pull request reviews.

Finally, the decision to prioritise architectural flexibility over immediate feature completeness was also a quality related resource decision. Rather than using the available time to implement a narrow set of isolated features, the team invested in a modular architecture that could support future extensions. This reduced the completeness of some user facing functionality in the short term, but strengthened the long term maintainability and value of the system.

Overall, quality assurance and resource management were closely connected throughout the project. Maintaining quality required continuous balancing of time, scope and capacity, supported by prioritisation, reestimation and deliberate decisions about how team effort should be allocated.

5.2 What the Team Would Do Differently

Reflecting on the project as a whole, several core decisions and approaches have been identified that could have been improved or handled differently. The first major reflection regards the use of WinUI. Though the framework provided plenty of documentation and tools for the team to utilize in the learning process, many of the frontend tasks proved more tedious than anticipated. A simpler framework would streamline frontend progress and reduce limitations connected to WinUI's use of XAML, a declarative language by nature. The occurrence of bugs or other limitations resulting from WinUI being a relatively new framework could also have been avoided. The team's preference for working with Visual Studio Code rather than Visual Studio was also highlighted.

Another essential reflection concerns the implementation of a more plugin oriented approach to architecture. As the project progressed, planned features such as context and screenshot capture proved to introduce a high degree of complexity to the system. A pluggable architecture would allow these features to be treated as independent .dll extensions, rather than being embedded into the core application. These features could then be loaded dynamically at runtime, resulting in a stronger modular approach in the application. Although a fully pluggable architecture was not implemented within the scope of this project, introducing clearer extension points and architectural separation in the development process would likely have improved flexibility, maintainability, and scalability; especially long-term.

5.3 Key Learning Outcomes

The bachelor's project has taught the team a multitude of lessons on a range of different topics. Evidently, the team has gained thorough knowledge of Kristiansand's planning and building department, related infrastructure and caseworker work processes. Additionally, team members have gotten the chance to explore and utilise new technologies and frameworks throughout the product development. Lastly, the project has highlighted the importance of project and process organisation, teamwork, communication and resource management as success factors for project-based work. The latter topics are expanded on below, as they summarize the most valuable learning outcomes for the team as a whole.

5.3.1 The Importance of Structure

Executing a project successfully requires a clear framework for planning, decision-making, and delivery. Going into the project, the program supervisors at the department had set up an external SAFe inspired structure for all the teams to follow. Executing a project according to such a structure was unfamiliar and took time to get used to. Setting goals for PI deliveries with minimal knowledge on the road ahead was difficult, and settling into a good rhythm took time.

While the team had prior experience with Scrum, the larger scope and higher expectations of professionalism pushed the team to engage with its underlying principles more deliberately than before. Section 5.1 describes the concrete practices that emerged from this learning. Although the program provides some external structure, the teams operate autonomously and are expected to manage internal structure, planning, productivity and delivery independently.

5.3.2 The Importance of Resource Management

As mentioned priorly, a project with a large scope and limited timeframe inevitably leads to discussions and decisions regarding how time and effort are distributed. Starting from scratch and relying on unfamiliar technologies for development means that everyone cannot learn everything, and work must be split between team members. On the other hand, the team is required to make architectural decisions and plan the road ahead together. Distributing and managing tasks and knowledge has been a recurring challenge throughout the project. The key learning is that capacity planning requires explicit accounting for research time, dependencies and coordination overhead, especially when working with new technology. Similarly, specialization creates efficiency, however it also requires deliberate practices in order to share knowledge to prevent silos, which block progress. These are lessons the team will carry forward into future collaborative work.

5.3.3 The Importance of Teamwork

Effective teamwork requires deliberate coordination and cooperative problem-solving. The project demonstrated that technical expertise *alone* is insufficient. Successful delivery depends equally on the team's ability to communicate effectively, share knowledge across boundaries, and surface obstacles early. The Scrum master role highlighted an unforeseen challenge: balancing facilitation responsibilities with individual development work. The

FoU-program's emphasis on this role, through dedicated forums, reinforced its importance for team dynamics, but also revealed the significant capacity it requires. The balance between team-level coordination and individual contribution was a recurring theme throughout the project.

In short, the project reinforced that collaboration is not automatic but requires intentional practices and continuous adjustment. The team's capacity to deliver was shaped as much by how members worked together, as by what each individual contributed. The acronym TEAM, standing for “together everyone achieves more” proved to be more than a slogan from the tech-lead, it was a practical requirement for success.

6. Conclusion

The purpose of this project was to explore how a context aware AI assistant could support municipal caseworkers in their everyday work at Kristiansand Municipality's planning and building department. The project addressed a real problem domain where caseworkers must navigate complex cases, fragmented information sources and time consuming manual processes. Through interviews, workflow mapping and iterative design, the team identified opportunities for AI support within information retrieval, document handling, case organisation and regulatory interpretation.

The delivered prototype demonstrates the core direction of such a system. It provides a Windows based desktop client connected to a backend architecture that supports chat based interaction, case management, document handling and communication with an AI service. While the prototype does not implement all intended functionality, it establishes a functional foundation for further development. The system shows how a caseworker oriented AI assistant can be structured around cases, context and conversational support, rather than functioning as a general purpose chatbot detached from the user's workflow.

A central outcome of the project is the architectural foundation that has been established. Given the limited timeframe and broad scope, the team prioritised flexibility, modularity and long term adaptability over complete implementation of every planned feature. This resulted in an architecture designed to support future integration of data sources, specialised agents, orchestration logic and additional context mechanisms. Some user-facing features were

therefore postponed, but this was a conscious tradeoff intended to increase the value of the project for future teams and continued development within the FoU 2026 program.

The project also provided significant learning outcomes related to process, collaboration and resource management. Working with unfamiliar technologies required continuous research, reestimation and adjustment of priorities. The team learned that capacity planning must account for technical uncertainty, coordination overhead and dependencies between frontend, backend and architecture work. Over time, more structured planning, subgroup discussions, written summaries and collaborative reviews improved the team's ability to distribute work while maintaining a shared understanding of the system.

Future development should focus on strengthening the connection between the assistant and real municipal data sources. Direct integration with Plan & Build 360, GeoDoc and other relevant systems would reduce the need for manual context providing and make the assistant more useful in realistic caseworker workflows. Further work should also prioritise retrieval based context handling, document processing, specialised agents, improved source references and broader user testing across different caseworker roles.

Overall, the project demonstrates that AI supported casework assistance is both technically feasible and relevant for municipal caseworkers. The prototype, architecture and documented recommendations provide a foundation that future teams can continue to build upon. Although the final product remains an early version, the project has contributed meaningful insight into how local AI solutions can be designed around public sector workflows, user needs and long term system adaptability.

References

Agile Business. (2014, October). Chapter 10: MoSCoW Prioritisation. Agile Business Consortium. Retrieved 03.04.26 from:

<https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>

Amazon (2023, 28. November). *What is RAG (Retrieval-Augmented Generation)?* AWS.

Retrieved 06.04.26 from: <https://aws.amazon.com/what-is/retrieval-augmented-generation/>

Amazon (2022, 6. April). *What is OCR (Optical Character Recognition)?*. AWS. Retrieved 06.04.26 from:

[https://aws.amazon.com/what-is/ocr/?ams%23interactive-card-vertical%23pattern-data--1819409748.search=What%20is%20OCR%20!\(Optical%20Character%20Recognition!\)%3F](https://aws.amazon.com/what-is/ocr/?ams%23interactive-card-vertical%23pattern-data--1819409748.search=What%20is%20OCR%20!(Optical%20Character%20Recognition!)%3F)

Atlassian. (n.d.). *What is DevOps?* Retrieved 05.05.26 from

<https://www.atlassian.com/devops>

Brattli, H., Garmann-Johnsen, N. F., & Utne, A. (2023). Design thinking på norsk. Cappelen Damm.

Microsoft (2024, 10. September). *Model-View-ViewModel (MVVM)*. Microsoft. Retrieved

03.04.26 from: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>

Microsoft. (n.d.). *The project triangle*. Microsoft. Retrieved 22.04.26 from:

<https://support.microsoft.com/en-us/office/the-project-triangle-8c892e06-d761-4d40-8e1f-17b33fdef810>

Yablonski, J. (n.d.). *Jakob's Law*. Laws of UX. Retrieved 03.04.26 from:

<https://lawsofux.com/jakobs-law/>

Appendix

Appendix A: Project Description

A1: Project Description Part 1

Bachelor project proposal Thesis information (Working) Title of the project: <i>Context-Aware AI Assistant for Municipal Caseworkers – Client and Backend Architecture.</i> Time frame of the thesis: <i>Spring semester 2026 (Bachelor level, UiA Kristiansand)</i> Supervisor: <i>University of Agder</i> – Department of Information systems Municipal partner: <i>Kristiansand Municipality,</i> Department of Plan and Build Company intro Kristiansand Municipality is among Norway's most digitally advanced municipalities, with a long-term commitment to research and development in artificial intelligence (AI), data management, and urban planning technology. The municipality collaborates with other public institutions such as Trondheim Municipality, KS, and DiBK on applied AI use cases, including document interpretation, zoning plan accessibility, and process automation in the public sector. This bachelor project aims to continue that innovation effort by exploring how AI assistants can be integrated into the everyday workflow of municipal caseworkers. The project will be developed in collaboration with the municipality's internal AI-lab and will build on ongoing research projects, but without requiring access to sensitive or restricted data.	 Project description The proposed project can be approached as a single integrated bachelor project or divided into two complementary sub-projects depending on student interests and available capacity. Part 1: Context-Aware Client Application (Windows / Cross-Platform) Develop a lightweight, context-aware client for caseworkers. Essentially a digital assistant capable of capturing the user's active work environment and relaying that context to an AI backend for assistance. <u>Key functional goals:</u> • Detect active applications, document titles, window focus, and relevant metadata via official Windows APIs (and possibly cross-platform alternatives). • Allow the user to trigger AI assistance via hotkey, capturing context and optional screenshots (user-provided or automated). • Transmit this contextual information to a backend endpoint (see Part 2) for processing and receive AI-generated responses. • Support a modular architecture enabling future functionality such as task automation or embedded mini-LLMs for local OCR/image processing. • Include a plugin framework so that new features or integrations can be added independently of the core application. • Prioritize usability, low system load, and easy deployment to municipal devices. Technology considerations: Preferred technologies include C# / .NET with Visual Studio for Windows-native integration, though cross-platform approaches (e.g., Electron, Python + Qt, or Flutter) may also be explored if suitable.
--	---

A2: Project Description Part 2

 Part 2: Backend / Orchestrator / Integration Architecture Design and implement a minimal viable backend service that serves as the bridge between the client and the municipality's AI infrastructure. This component should orchestrate data exchange with one or more large language models (LLMs) and connected tools through the Model Context Protocol (MCP). <u>Key objectives:</u> • Establish a modular backend that can route context data from the client to appropriate AI/ML services • Explore lightweight orchestration methods that support multiple specialized agents (e.g., retrieval agents for zoning data, document search, or archive lookups) • Ensure a simple and clear API boundary between the client and backend • Investigate how open standards and container-based deployment (Docker, FastAPI, etc.) can make the system portable and maintainable across municipal and cloud environments • Prototype a small-scale MVP that demonstrates end-to-end functionality between the client and backend Technology considerations: Python is recommended due to strong ecosystem support for AI, APIs, and orchestration frameworks (LangChain, FastAPI, etc.), but other languages are acceptable. The architecture should avoid dependency on proprietary cloud services; solutions should be open, flexible, and cost-efficient. Expected Deliverables: • For Part 1: A functioning context-aware desktop client with plugin-ready architecture. • For Part 2: A lightweight backend MVP that can receive contextual data and communicate with one or more LLMs through MCP or equivalent mechanisms. • Documentation covering architectural choices, API interfaces, and data flow. • Recommendations for scalability, future functionality (e.g., speech, automation), and security considerations.	 Learning Outcomes Students will gain hands-on experience with: • Application architecture and modular design • Integration of local clients with AI/ML backend services • Working with modern APIs, protocols (such as MCP), and system interoperability • Understanding real-world municipal challenges and constraints around data access, privacy, and usability • Agile collaboration with a public-sector partner Openness and Adaptability This proposal is a draft and may be adjusted in collaboration with UiA supervisors and students. Kristiansand Municipality is open to alternative approaches, including: • Expanding the focus to web-based or mobile interfaces • Exploring hybrid local/cloud deployment models. • Integrating additional municipal data sources or AI agents if feasible within scope. The final structure may consist of one joint bachelor project or two coordinated sub-projects, depending on student group size and skill distribution. Rights and Open Collaboration As a public-sector organization, Kristiansand Municipality believes that research and innovation carried out in collaboration with universities should benefit the public, other municipalities, and the wider Norwegian digital community. Therefore, we encourage that all project results, source code, documentation, or prototypes, be made available under open-source licenses whenever feasible. This ensures that future students, municipalities, and developers can continue building upon the work, amplifying its societal value. At the same time, we recognize and support the students' ownership of their own work. Students are free to reuse, further develop, or commercialize the results of the project after submission, provided that Kristiansand municipality retains a royalty-free right to use the future deliverables for internal or public-sector purposes. We believe this approach promotes transparency, innovation, and shared value creation, aligning with both academic openness and the municipality's commitment to strengthening digital collaboration across the Norwegian public sector.
---	---

Appendix B: Analysis

B1: Sprint 1 Interview Questions

1. Hvordan ser arbeidsprosessen ut?
2. Hvilken del av arbeidsprosessen er det dere bruker mest tid på?
3. Hvilke type dokumenter forekommer? Hva slags funksjonalitet er dere ute etter?
4. Hvordan er oppsettet på filsystemet og hvilke deler av filsystemet er folk inne på mest?
5. Hvordan er dokumentene til kommunen lagt opp?
6. Hvilke type roller bruker systemet?
→ Hva er viktig for disse/de ulike rollene?
7. Kan vi få screenshots?
8. Hvis du kunne endre på én ting i systemet, hva ville det vært?
9. Hvor ofte må dere lage workarounds? (midlertidige løsninger)
10. Hvordan er samarbeidet med andre roller? Hvilke typer henvendelser er mest normale?
11. Er det noen relevante lovverk ifht feks. personvern vi må ta i betraktning?
-> Eksempelvis at dokumenter med sensitiv informasjon ikke kan bli brukt av LLMer fra private bedrifter som ChatGPT
12. Hva er dine erfaringer med AI og LLM? Positive aspekter og smertepunkter?
13. Er det noe annet vi ikke har spurt om, som du vil nevne?
14. Er det noen andre du tenker det kan være greit for oss å intervju? (henvis oss)
 - Er du villig til å bli med på et oppfølgingsintervju, ved behov? Og når evt passer det?

B2: Sprint 2 Interview Questions

Intervjuspørsmål v2

1. Hva går jobben din ut på?
2. Hvordan ser arbeidsprosessen din ut? Hva slags (interne/eksterne) dokumenter / ressurser forekommer?
3. Hva skiller de ulike rollene; ressurser, arbeidsflyt? Er arbeidsprosessen lik i avdelingene? Hvordan fungerer avdelingene innenfor arkiv og planregister?
4. Opplever du noen frustrasjoner / smertepunkter ifht prosessen, i så fall hvilke?
5. Hvordan fungerer samarbeidet mellom avdelingene?
6. Har du noen funksjonalitet du savner / ønsker i prosessen?
7. Har du mulighet til å gi screenshots/screenvideo av saksbehandler-programmet (Plan and Build 360) dere bruker?

B3: Supervision & Illegality Caseworker Workflow and Pain Points

Steg	Aktiviteter & Smertepunkter (Tilsyn & Ulovlighetssaker)
1. Motta bekymringsmelding	Bekymringsmelding fra publikum eller brannvesen fordeles av leder via Plan & Build 360. Saksbehandler gjennomgår meldingen og identifiserer sakens art <i>⚠ Ingen strukturert oversikt over hvem som melder og hvilken type bekymring som foreligger → vanskelig å vurdere hastegrad før man har gått inn i saken.</i>
2. Sak & historikk	Åpner saken i Geodoc og Plan & Build 360 for å hente historikk på eiendommen, matrikelopplysninger, boenheter og eiere. <i>⚠ Innhenting av historikk fra Geodoc er tidkrevende og kan ende opp mangelfull. (Plan & Build 360 har mangler i integrasjon mellom sakstyper, noe som skaper duplikater og vanskeliggjør navigering.)</i>
3. Vurdere ulovlighet	Gjennomgår bekymringsmeldingen mot gjeldende forskrifter (TEK17, Byggeforskriften 1985) hentet fra Byggforsk og Lovdata. Vurderer om ulovlighet er sannsynlig, ved ja: - Sender enten brev til hjemmelshaver om tilbakemelding - Åpner tilsynssak <i>⚠ Riktig forskrift avhenger av bygghdato og må identifiseres manuelt. Tolkningstvil mellom forskrifter kan stoppe fremdriften.</i>
4. Stedlig tilsyn	Avvik / Åpning av tilsynssak: to saksbehandlere reiser ut på lokasjon, brannvesen ved behov. Kontrollerer brannskiller, rømningsveier og boenheter. Skriver tilsynsrapport med alle funn. Så går saken videre til oppfølging og vedtak. Uten avvik lukkes saken. <i>⚠ Alltid to saksbehandlere krever koordinering.</i>
5. Oppfølging & vedtak	Etter at avvik er dokumentert i tilsynsrapporten, sendes forhåndsvarsel om pålegg om retting (fysisk fjerner eller endrer det ulovlige forholdet) til hjemmelshaver (juridiske eieren av eiendommen) med en satt frist . Det videre forløpet avhenger av hjemmelshavers respons: Retting ikke gjennomført innen fristen: formelt vedtak om pålegg med ny frist fattes. Byggesøknad innsendt: ulovlighetssaken settes på pause til byggesaken er ferdigattestert. Vedtak sendes til berørte parter. <i>⚠ Gjentakende og feilplasserte prosesser i Plan & Build 360. Manglende oversikt gjør at saksbehandlere fører parallell saksoversikt i Excel med fargekoder og kommentarer.</i>
6. Gebyr & avslutning	Ved fortsatt regelbrudd vurderes overtredelsesgebyr basert på alvorlighetsgrad og veileder. Forhåndsvarsel med klagerett sendes før formelt vedtak. Saken lukkes når forholdet er rettet eller prosessen er avsluttet. <i>⚠ Manuell gebyrberegning</i>
7. Klagebehandling	Dersom hjemmelshaver klager, utarbeides saksfremlegg og dokumentasjon til Statsforvalteren. <i>⚠ Klagebehandling er tidkrevende: krever omfattende skriving av saksfremlegg for hver enkelt sak.</i>

B4: Caseworker Persona 1

Navn	Vilde Andersen
Alder	42 år
Stilling	Saksbehandler, Plan og bygg
Erfaring	14 år i kommunal saksbehandling
Fagbakgrunn	Bachelor i offentlig administrasjon
Arbeidssted	Kristiansand kommune
Saksvolum	8–15 saker per uke
Verktøy	Plan & Build 360, GISLine, Arealplaner.no, DiBK, NVE-kart, Naturbase, Askeladden

"Jeg bruker mesteparten av dagen på å lete etter informasjon som burde vært lett tilgjengelig."

Utfordringer

- Manuelt arkivsøk med nøkkelord for å finne lignende saker
- Tolking av gamle reguleringsplaner med utdatert terminologi
- Må bytte mellom mange systemer (GISLine, DiBK, Lovdata) for én sak
- Vanskelig å finne relevante §19-2-vedtak og direktoratsuttalelser
- Koordinering med Vegvesen, statsforvalter og Kystverket tar lang tid
- Gebyrberegning og matrikkelføring gjøres manuelt og separat

Mål og ønsker

- Rask tilgang til relevante tidligere vedtak og lignende saker
- Trygghet i tolkning av plan og lovverk
- Ensartet og faglig solid argumentasjon i vedtakene
- Effektiv koordinering av uttalelser og frister

B5: Scenario 1 - Persona Workflow without Application

Vilde mottar en søknad om dispensasjon (§19-2) for tilbygg til enebolig i et eldre reguleringsplanområde. Tiltaket overstiger tillatt utnyttelsesgrad, og søker ber om unntak. Vilde må vurdere om vilkårene for dispensasjon er oppfylt.

Steg	Aktivitet	Beskrivelse og Smertepunkter
1	Åpne søknad	Åpner søknaden i Plan & Build 360 og blar manuelt gjennom tegninger, situasjonsplan og følgebrev. Noterer selv hva søknaden gjelder. <i>⚠ Ingen strukturert oversikt: usikker på om alle vedlegg er komplette.</i>
2	Kart & eiendom	Åpner GISLine, NVE-kart, Naturbase og Askeladden separat. Noterer funn i eget dokument. <i>⚠ Fire systemer for én eiendom: risiko for å overse relevante opplysninger.</i>
3	Plan & lovverk	Reguleringsplanen er fra 1987 med utdatert terminologi. Slår opp i Arealplaner.no, DiBK og Lovdata. Sender e-post til kollega for dobbeltsjekk, venter til neste dag. <i>⚠ Tolkningstvil stopper fremdriften. Saken mister en halv dag.</i>
4	Høringer og uttalelser	Husker at 100-metersbelte mot sjø kan kreve statsforvalter-uttalelse. Ser gjennom interne prosedyrer og koordinerer per e-post. <i>⚠ Usikkerhet om riktige instanser. E-postkoordinering er tidkrevende og vanskelig å spore.</i>
5	Fatte vedtak	Søker manuelt i arkivet etter lignende §19-2-saker. Skriver dispensasjonsvurderingen fra bunnen av. <i>⚠ Ingen strukturert hjelp til å finne sammenligningssaker. Argumentasjon varierer fra sak til sak.</i>
6	Matrikkelføring	Tar skjermbilde av matrikkelinformasjon og sender e-post med lenke til vedtaket til en matrikkelfører. <i>⚠ Manuell og todelt prosess: lett å gjøre feil eller glemme steg.</i>
7	Gebyrberegning	Beregner gebyret manuelt basert på ressursbruk og sakens innhold. Sender e-post til fakturaseksjon. <i>⚠ Ingen standardmål: utsatt for feil og inkonsistens på tvers av saker.</i>

Resultat

Saken bruker 2–3 dager. Vilde er usikker på om dispensasjonsargumentasjonen er sterk nok, og om hun har funnet de mest relevante sammenligningssakene. Vedtaket sendes, men prosessen har vært fragmentert og tidkrevende.

B6: Caseworker Persona 2

Navn	Vilde Andersen
Alder	42 år
Stilling	Saksbehandler, Plan og bygg
Erfaring	14 år i kommunal saksbehandling
Fagbakgrunn	Bachelor i offentlig administrasjon
Arbeidssted	Kristiansand kommune
Saksvolum	8–15 saker per uke
Verktøy	AI-assistent (primær), Plan & Build 360, GISLine

"Jeg slipper å lete i fem systemer for å finne noe jeg vet finnes, og kildene er rett der så jeg kan sjekke selv."

Utfordringer

- Stoler ikke blindt på AI - dobbeltsjekker kritiske tolkningskonklusjoner
- Matrikkelføring i GISLine er fortsatt manuell
- Ny arbeidsflyt krever tilvenning

Mål og ønsker

- Bruke AI til å finne relevante §19-2-vedtak raskt og strukturert
- Få kontekstuell tolkehjelp på plan og lovverk med kildehenvisning
- Mer ensartet argumentasjon på tvers av saker
- Raskere koordinering med riktige instanser foreslått automatisk

B7: Scenario 2 - Persona Workflow with Application

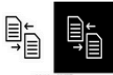
Samme sak som over: dispensasjon (§19-2) for tilbygg i eldre reguleringsplanområde. Vilde bruker AI-assistenten som støtteverktøy gjennom hele prosessen.

Steg	Aktivitet	Beskrivelse
1	Åpne søknad	AI-assistenten oppsummerer dokumentasjonen og lister opp tegninger, følgebrev og situasjonsplan. Varsler om eventuelle mangler. <i>✓ Rask oversikt - identifiserer mangler uten manuell gjennomgang.</i>
2	Kart & eiendom	Assistenten samler data fra Proffkart, GISLine, NVE, Naturbase og Askeladden i én rapport. <i>✓ Komplett kartrapport på sekunder - ingen risiko for å glemme en kilde.</i>
3	Plan & lovverk	Vilde markerer uklart punkt i planen. Assistenten forklarer med gjeldende TEK og PBL med kildehenvisning til DiBK og Lovdata. <i>✓ Tolkningstvil løst på minutter - med presis kilde hun kan verifisere selv.</i>
4	Høringer og uttalelser	Assistenten foreslår statsforvalter (100-metersbelte mot sjø) og Kystverket, med begrunnelse og hjemmel. <i>✓ Ingen usikkerhet om instanser - koordinering starter umiddelbart.</i>
5	Fatte vedtak	Assistenten henter §19-2-vedtak sortert etter: 1) samme reguleringsplan, 2) samme område, 3) samme tema (likhetsprinsippet). Foreslår dispensasjonstekst med kildehenvisning. <i>✓ Strukturert sammenligning - sterkere og mer ensartet argumentasjon.</i>
6	Matrikkelføring	Assistenten strukturerer matrikkelinformasjon fra vedtaket klar for overføring. Vilde sender til matrikkelfører. <i>✓ Raskere forberedelser - informasjon klar og strukturert.</i>
7	Gebyrberegning	Assistenten foreslår gebyr basert på ressursbruk og lignende saker. Vilde vurderer og justerer ved behov. <i>✓ Standardisert grunnlag - raskere og mer konsistent beregning.</i>

Resultat

Saken behandles på én dag. Vilde har høy faglig trygghet, argumentasjonen er godt forankret i tidligere praksis, og hun bruker tid på vurdering fremfor informasjonsinnhenting. Vedtakene holder jevnere kvalitet på tvers av saker.

B8: User Journey

Reisens steg:	1. Åpne søknad	2. Kart & eiendom	3. Plan & lovverk	4. Høringer og uttalelser	5. Fåtte vedtak	6. Matrikkelføring	7. Gebyrberegning
FOR Å:	Forstå hva som søkes om og være forberedt på saken	Vite alt om eiendommen	Vite hva som gjelder juridisk	Få nødvendige spesialistvurderinger	Gi korrekt, begrunnet vedtak	Fullføre lovpålagt matrikkelløp	Beregne riktig gebyr for saken
VIL KUNDE:	Identifisere alle dokumenter, forstå innholdet og ha oversikt over lignende saker	Sjekke eiendomsgrenser, matrikkel, flom/skred, kulturminner	Forstå reguleringsplan, kommuneplan, TEK, PBL	Koordinere med interne instanser (ingeniørvesen, parkvesen) og eksterne instanser (Statsforvalter, Vegvesen, Kystverket m.fl.)	Vedtak føres i Plan & Build og sendes videre til nabo/statsforvalter ved behov. Vurdere dispensasjon (§19-2) med argumentasjonshjelp fra LLM.	Registrere vedtak i matrikkel via GISLine	Beregne og sende gebyr til fakturaseksjon
INTERAKSJONER (med tjenesten) Illustrasjon av de viktigste brukerhistoriene langs brukerreisen							
Handlingsbeskrivelse	Åpner søknaden i Plan & Build og gjennomgår dokumentasjonen (tegninger, følgebrev, situasjonsplan). Argumenter hentes fra saksbehandlerens egne vedtak – ikke fra tiltakshaver.	Søker opp adressen i valgte kartverktøy og vurderer hvilket plangrunnlag som gjelder	Gjennomgår ulike planer og lovverk relevant for saken (TEK og PBL)	Gjennomgår kontroll mot flere eksterne løsninger: NVE, Naturbase, Askeladden. Ved berørelse av spesielle fagområder, sendes den på uttalelse (intern eller eksternt).	Dersom søknad ikke er i tråd med plan, gjennomføres dispensasjonsvurderingen (§19-2). Nye vedtak punktfestes geografisk i kartet og knyttes til reguleringsplanen. Vedtak føres inn i Plan & Build.	Matrikkelinformasjon inkluderes i vedtaket. Skjermbilde sendes per e-post med lenke. Matrikkelføring gjøres av egen matrikkelfører i GISLine.	Gebyr beregnes separat fra matrikkelføring, basert på ressursbruk og sakens innhold. Sendes til fakturaseksjon.
UBESVARTE SPØRSMÅL:	Er alle dokumenter komplette? Hvilke tegninger viser hva? Finnes lignende tidligere saker og hva ble konklusjonen?	Stemmer grensene? Flomfare? Skredfare? Kulturminner i nærheten?	Hvordan tolkes denne gamle planen? Hva sier TEK og PBL om dette?	Hvem må uttale seg? Hva er fristene? Har andre kommuner møtt dette?	Oppfylles saken vilkår for dispensasjon? Hva har tidligere praksis vært? Hvilke tidligere vedtak fra samme reguleringsplan/område er relevante?	Hvem skal kopi av vedtak (utenom søker, naboer, statsforvalter)?	Hva er riktig gebyrgrunnlag basert på ressursbruk og sakens innhold?
BARRIERER/ULEMPER:	Mange dokumenter per søknad gjør prosessen uoversiktlig og krever manuell gjennomgang, og lignende saker må finnes ved manuell nøkkelord-søk i arkivet.	Tungvint matrikkeloppslag i GISLine, må sjekke flere ulike kartkilder manuelt	Gammel plan-terminologi, må bytte mellom Arealplaner.no, DiBK, Lovdata. Kan oppstå tolkningstilv.	Tidkrevende å koordinere flere instanser, følge opp frister	Vanskelig å finne relevante tidligere §19-2 vedtak, utliggende direktivuttalelser. Mangler enkel tilgang til vedtak fra samme planområde/tema.	Manuell matrikkelføring i GISLine, prosessen er todelt og lett å blande med gebyr	Beregne gebyr selv i vedtak, sende manuell e-post til fakturaseksjon. Gebyrmodul kommer snart inn i Plan and Build
ØNSKEDE FORDELER:	Rask oversikt over dokumentasjon og tidlig avdekking av mangler, samt læring fra tidligere saker for å unngå feil.	Komplett oversikt til kartene på ett sted, ingen overraskelser	Trygghet i tolkning av plan og lov	Raskere saksgang, god koordinering	Riktig og godt begrunnet vedtak med referanser. Ensartet språk og argumentasjon på tvers av saker.	Effektiv og korrekt matrikkelløp	Effektiv og feilfri gebyrberegning
IDEER (hva kunne hjelpet?):	LLM oppsummerer dokumenter og lenker tegninger til beskrivelser. AI-drevet vektorsøk i arkiv for å finne lignende saker basert på semantisk likhet.	LLM henter data fra Proffkart, GISLine, NVE, Naturbase, Askeladden – samlet rapport	LLM svarer på spørsmål basert på Lovdata, DiBK. Evt. kontekstuell tolkehjelp: marker tekst → forklaring mot gjeldende lov.	LLM gir forslag til hvilke instanser saken bør sendes til og hvorfor (støttefunksjon). Eks: 100 m-belte mot sjø → statsforvalter, fylkesvei → kommune.	LLM henter §19-2-vedtak sortert etter: 1) reguleringsplan, 2) område, 3) tema (likhetsprinsippet). LLM gir argumentasjonshjelp – skriver ikke vedtak (føres i Plan & Build). Kildehenvisning vises.	LLM strukturerer og klargjør matrikkelinformasjon fra vedtaket. Selve matrikkelføringen gjøres alltid av matrikkelfører – AI erstatter ikke denne rollen.	LLM foreslår gebyr basert på ressursbruk og sakens innhold, som saksbehandler kan vurdere og justere.

B9: Caseworker Feedback on User Journey

Brukerreise tilbakemeldinger (*Punktene saksbehandler hadde innspill på*)

1. Åpne søknad

Hun presiserte at ved innhenting av argumenter¹ så skal informasjon bli hentet fra saksbehandlerens vedtak, ikke søknaden fra brukeren/tiltakshaver.

Dette sikrer at argumentasjonen er basert på kommunens egne faglige vurderinger og ikke bare gjenspeiler hva søkeren selv har skrevet. Som gir kvalitetssikring.

3. Plan og lovverk:

Funksjonalitet til LLM: Kontekstuell tolkehjelp: marker tekst → få forklaring mot gjeldende lov.

God nytte da dette er tidskrevende for saksbehandlere.

- Dersom man ikke rekker å implementere markeringsfunksjonaliteten, videreutvikler dette ved senere anledning?
- Alternativ funksjonalitet som er nåværende tanke: skriver et spørsmål på å få hjelp med å tolke lov og LLM gir et svar basert på relevant ressurser, eks lovdata

4. Høringer og uttalelser: Koordinering med Vegvesen, Kystverket, statsforvalter
Istedenfor “Automatisere henting av data mot eksterne ressurser”:

Funksjonalitet til LLM:

LLM gir et *forslag* til hvilke instanser en sak bør sendes til, og hvorfor

– altså som en støttefunksjon, ikke en fullautomatisert løsning. Tidsbesparende.

eks: 100 meters belte mot sjø → statsforvalter, eller fylkesvei → kommune

5. Vedtak

Det ble nevnt at nye vedtak punktfestes i kartet, altså knyttes geografisk til reguleringsplanen. Dette er relevant for hvordan vedtak presenteres og kobles til plangrunnlaget i systemet.

Funksjonalitet til LLM:

1. Hun presiserte også at LLM ikke trenger å skrive et direkte forslag til vedtak (da dette føres inn i Plan And Build), men bruke LLM for å innhente informasjon til å føre inn argumenter i vedtak.
2. LLM kan imidlertid komme med forslag til å skrive dispensasjon (etter forespørsel fra saksbehandler), med der LLM returnerer forslag i form av tekst.
 - **Dispensasjon/utnyttelsesgrad² (med mer):**
Prioritering på visning av liste:

1. Saker fra selve reguleringsplanen søknaden tilhører.
2. Saker fra samme område.

¹ Argumenter: **saklige, juridiske og planfaglige vurderinger for å begrunne et vedtak**

² **Utnyttelsesgrad:** regel/rammebetingelse for eiendommen.

Dispensasjon: unntaket fra regelen (inkludert utnyttelsesgraden).

3. Nye dispensasjoner innenfor samme tema, basert på *Likhetsprinsippet*³.

Hensikt: Tanken er at systemet trekker frem de mest relevante tidligere vedtakene, slik at saksbehandler kan bruke disse som grunnlag for sin vurdering.

- + Vedtaket kan måtte sendes videre til andre parter, eks nabo eller statsforvalter.

6. Matrikkelføring og Gebyr (dele det opp i brukerreise)

Saksbehandler understreket at dette punktet egentlig består av to helt forskjellige oppgaver som ikke bør blandes:

1. **Matrikkelinformasjon** (lovpålagt oppgave)

Nåværende flyt for saksbehandler: Matrikkelinformasjon tas med i vedtaket, tas et skjermbilde av dette, og det blir sendt som e-post til med lenke til vedtaket.

Selve matrikkelføringen gjøres deretter av en egen matrikkelfører (GISLine).

2. **Gebyrberegning:** Hun ga et konkret bruksområde for LLM-en:

Funksjonalitet til LLM: LLM kan komme med et forslag til gebyr etter ressursbruk og sakens innhold på forespørsel fra saksbehandler, basert på tidligere vurderinger fra matrikkel eller vedtak fra GeoDoc (?), som saksbehandlere så kan vurdere og eventuelt justere.

Tilbakemeldinger til UI

Generell funksjonalitet som er ønsket

Under gjennomgangen av sakssiden kom saksbehandleren med innspill på hvordan hun faktisk ser for seg å bruke et slikt system i hverdagen.

Det hun vil ha hjelp til er primært å *finne informasjon til vedtak*, særlig ved å skrive dispensasjoner og saksfremlegg til politikerne.

Eksempel arbeidsflyt:

1. Saksbehandler forteller LLM at vedtaket skal bli et avslag, og deretter får hjelp til å formulere argumentasjonen.
2. Hun ønsker også at systemet aktivt henter inn andre vedtak gjort i kommunen som kan brukes som referanse.

→ Fordelen med dette er at språk og argumentasjon blir mer ensartet på tvers av saker, noe som gir en mer profesjonell og enhetlig kommunikasjon utad.

³ *Likhetsprinsippet: i plan- og bygningsloven (pbl.) innebærer at like saker skal behandles likt, og at forvaltningen (kommunen) må treffe vedtak basert på objektive, saklige vurderinger*

Kildehenvisning på samtalsiden

Visning av kilde: Ved spørsmål om hvordan kildereferansen skal komme frem, var hun usikker på om det skulle vises i applikasjonen eller nettleser. Kom frem til at man kan prøve oss frem i applikasjonen, først og deretter prøve nettleser hvis dette føles unaturlig.

Kildehenvisning: Hun ville også ha mer konkret kildehenvisning (eksempelvis som i notebook.llm) der man viser nøyaktig hvor ressursen kommer fra (tidsbesparende når saksbehandler skal gå igjennom kilder).

Diskuterte hvordan man implementerer dette:

1. Sidevisning
2. Bytt vindu - så lenge man hadde mulighet til å få det opp i et nytt vindu for etter å ha trykket på ressurs
3. Faner

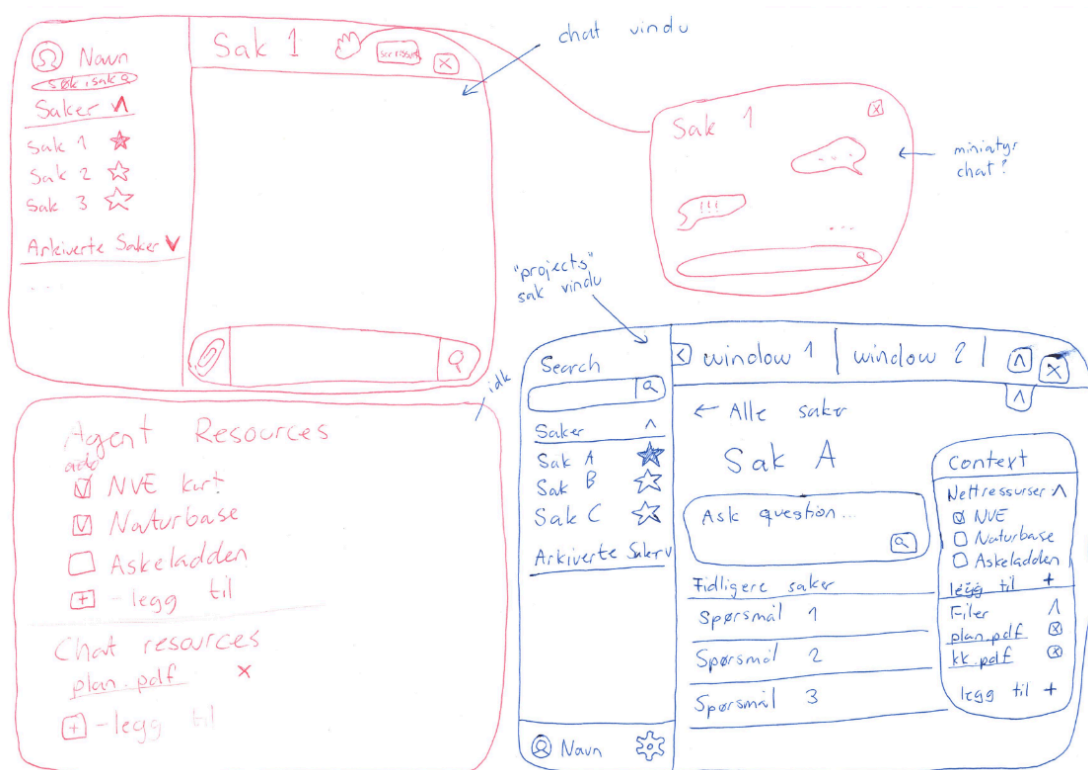
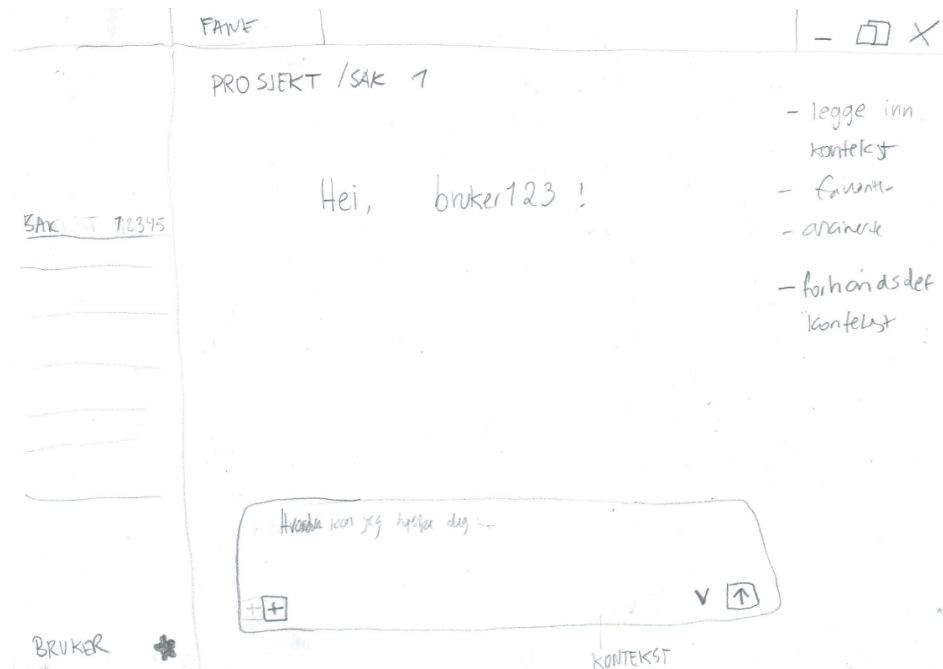
B10: MoSCoW Analysis Frontend

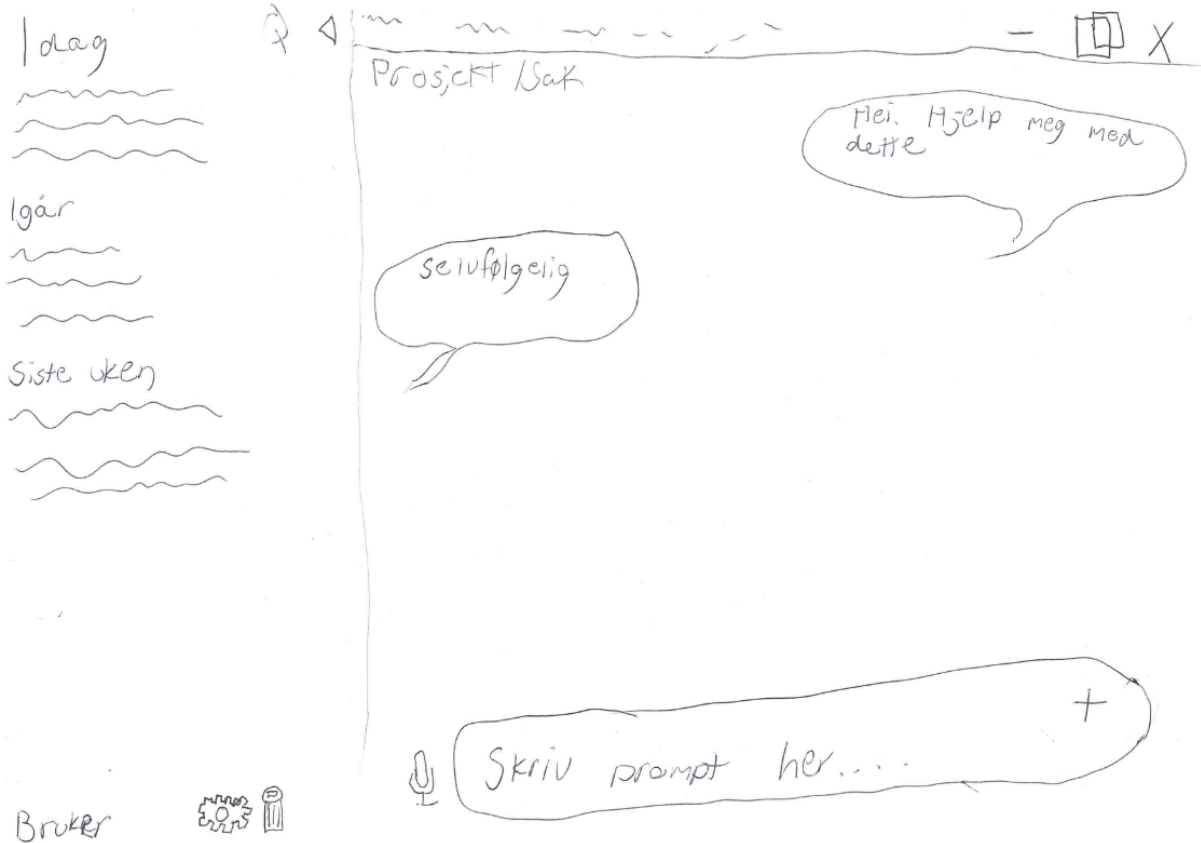
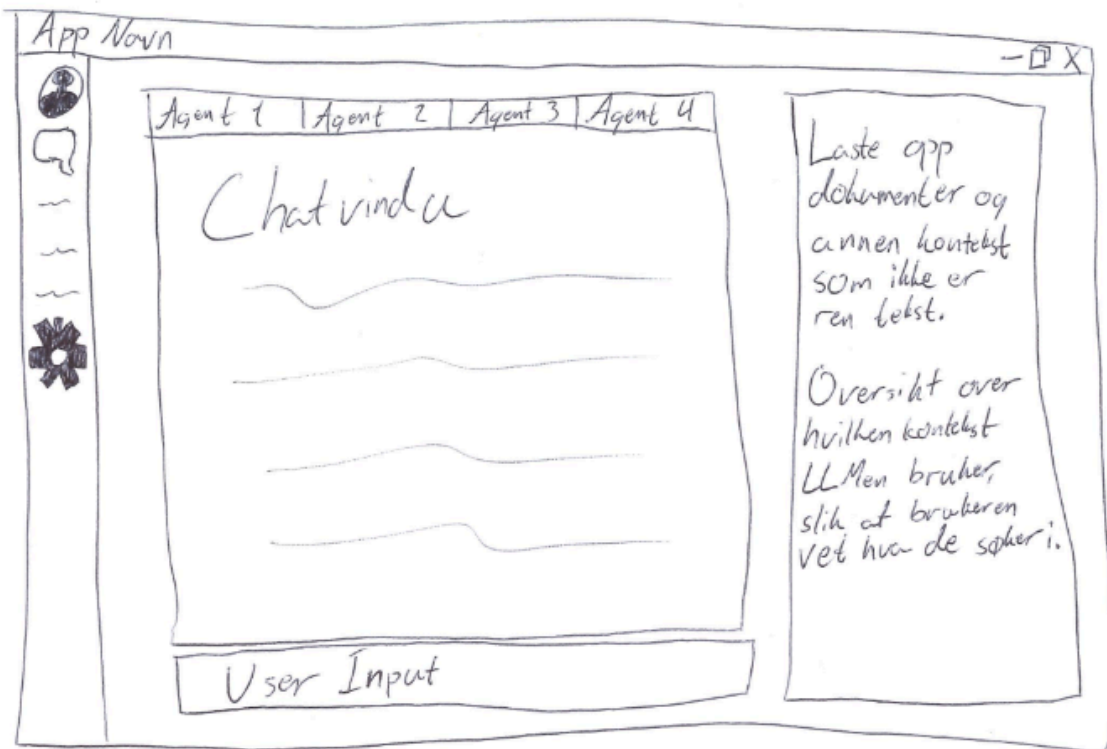
The frontend MoSCoW analysis bases itself on the findings of the analysis phase.

https://drive.google.com/file/d/1IcbO9w9EQPtpdf4VZb2qJob7-vvXvHaO/view?usp=drive_link

Appendix C: Design Material

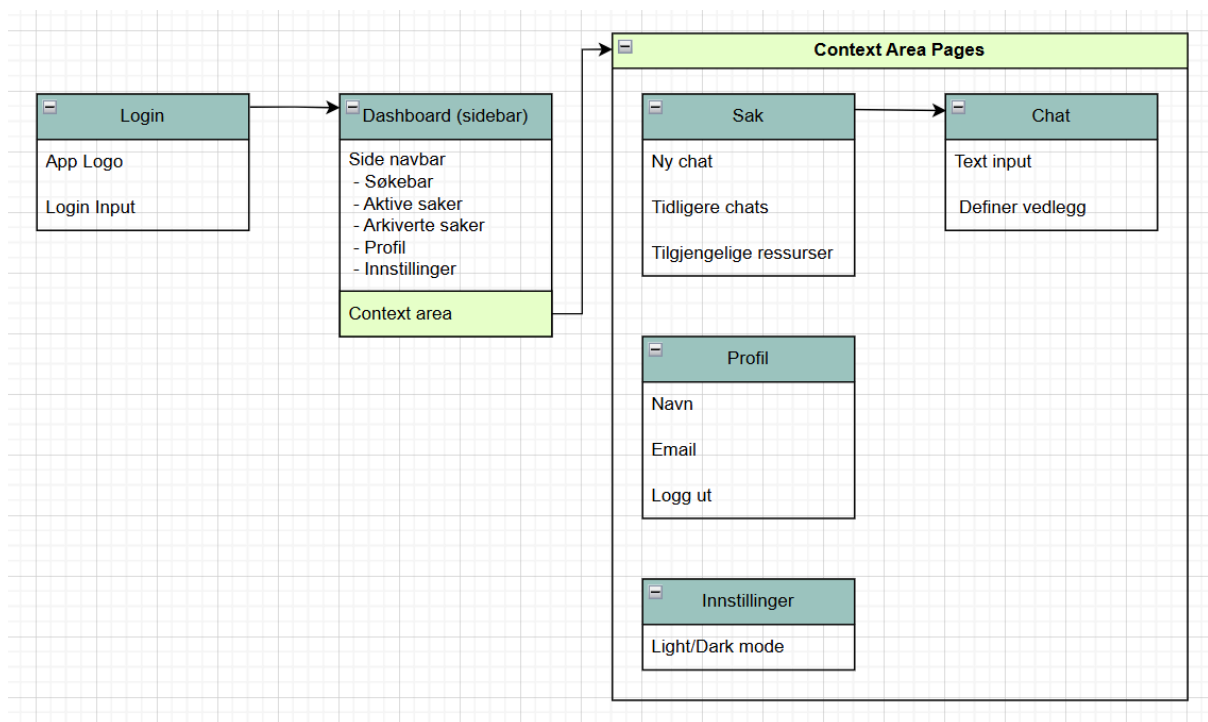
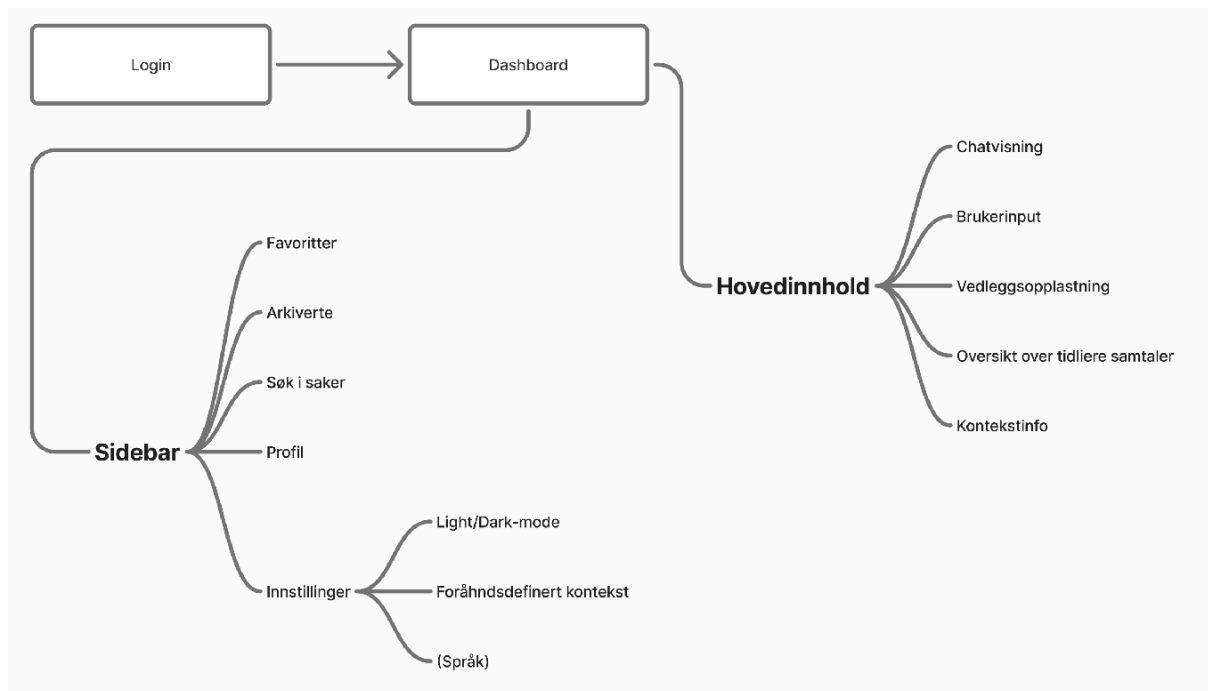
C1: Initial UI Sketches



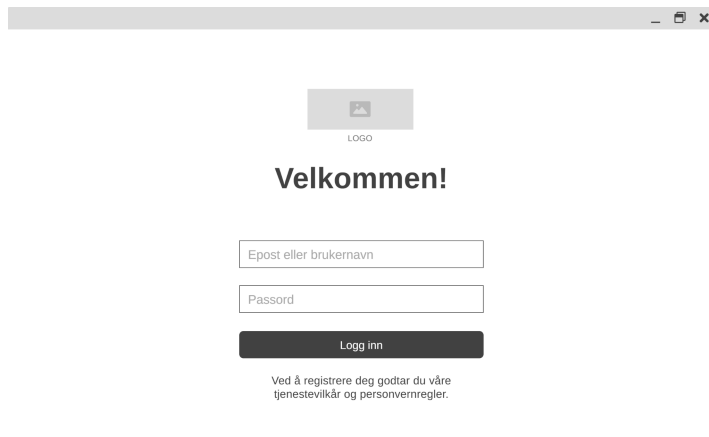




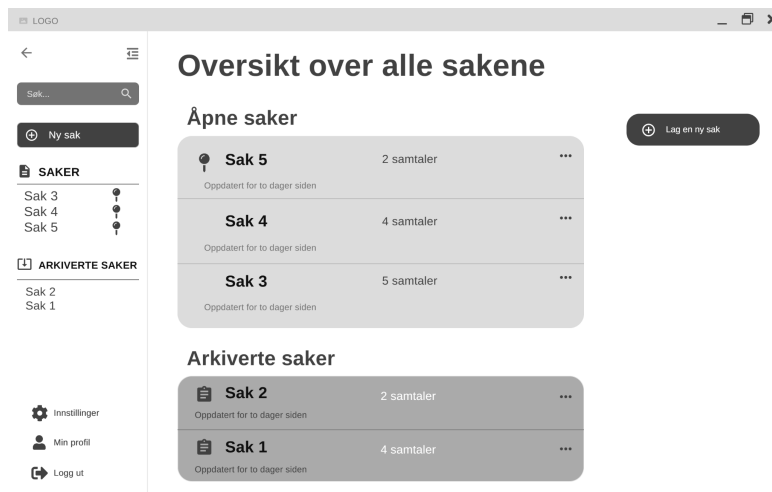
C2: UI Navigation and Planned Pages



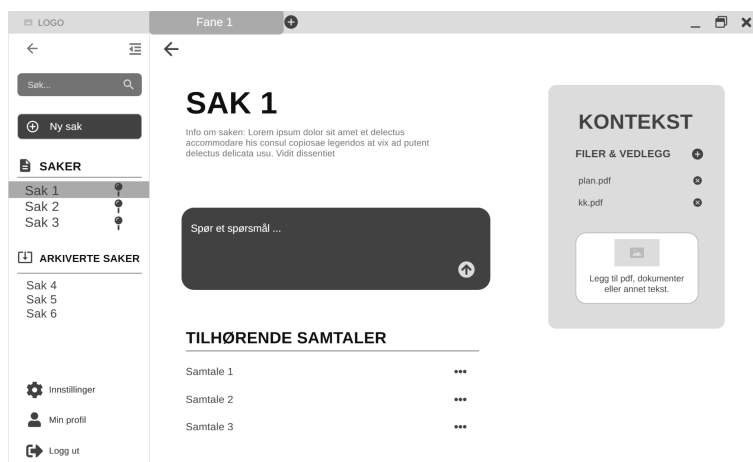
C3: Login Page Wireframe



C4: Case Overview Page Wireframe



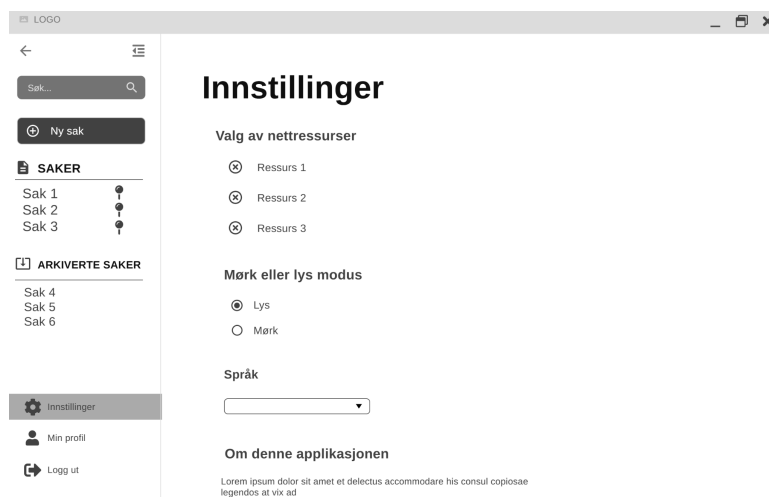
C5: Case Page Wireframe



C6: Chat Page Wireframe



C7: Settings Page Wireframe



C8: Profile Page Wireframe

LOGO

←

Søk...

Ny sak

SAKER

Sak 1
Sak 2
Sak 3

ARKIVERTE SAKER

Sak 4
Sak 5
Sak 6

Innstillinger

Min profil

Logg ut

Profil

Endre bilde

Endre fornavn
Skriv inn her

Endre etternavn
Skriv inn her

Endre passord
Skriv inn her

Lagre

Logg ut

C9: Create Case Page Wireframe

LOGO

←

Søk...

Ny sak

PÅGÅENDE SAK

Sak 1
Sak 2
Sak 3

ARKIVERTE SAKER

Sak 4
Sak 5
Sak 6

Innstillinger

Min profil

Logg ut

Opprette ny sak

Tittel
Gi et navn eller så vil det bli generert et automatisk

Beskrivelse
Vennligst gi en beskrivelse av saken

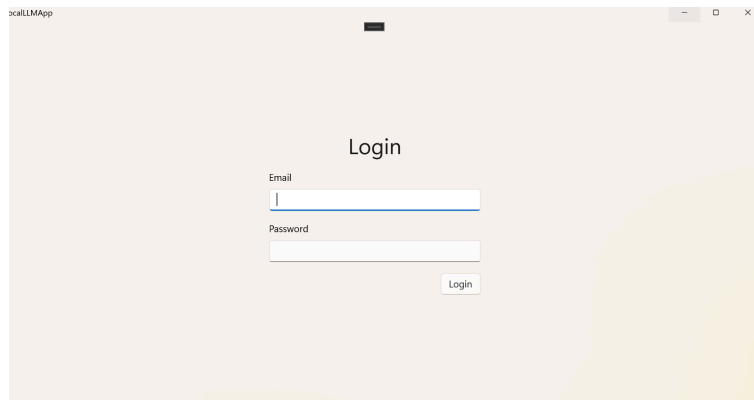
Vedlegg
Velg en fil eller dra og slipp en fil

Avbryt

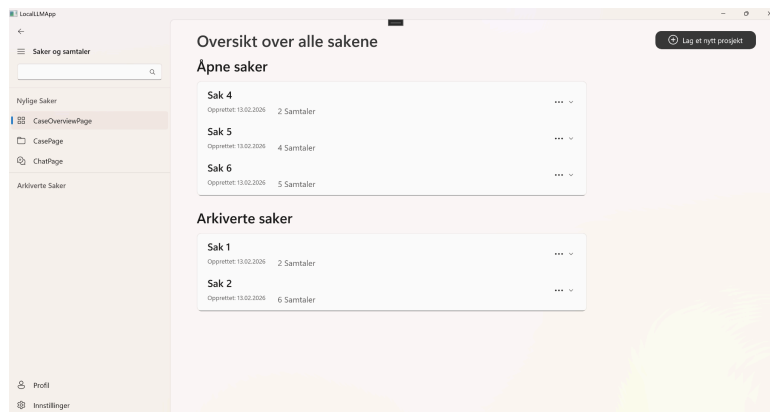
Opprett sak

Appendix D: WinUI Prototypes

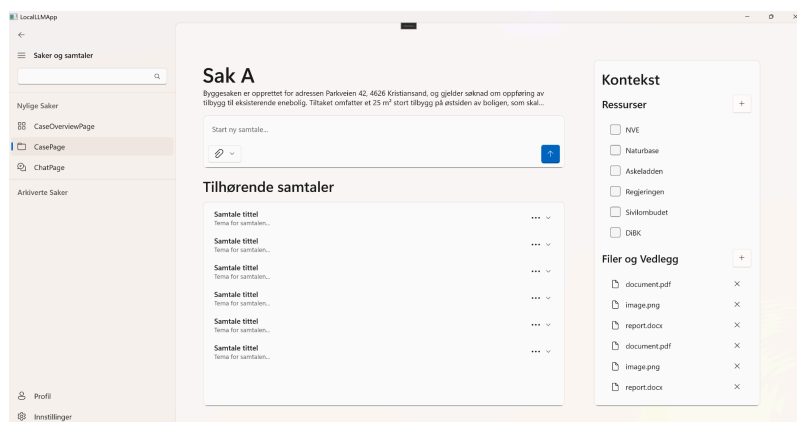
D1: Login Page Prototype



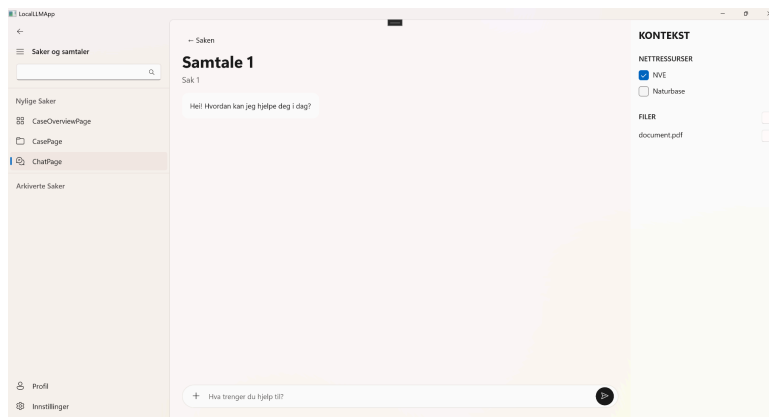
D2: Case Overview Page Prototype



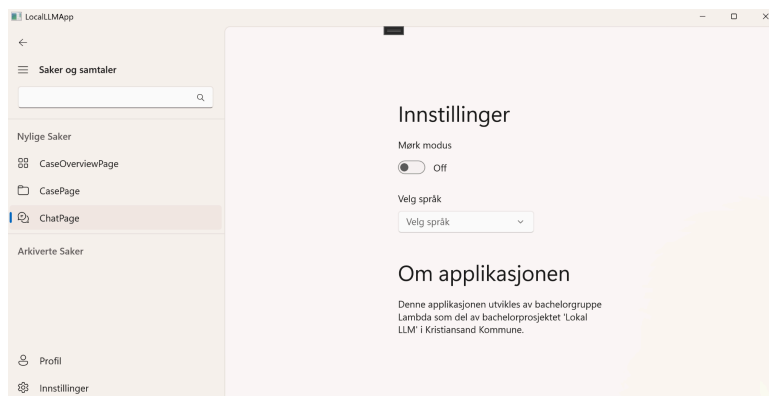
D3: Case Page Prototype



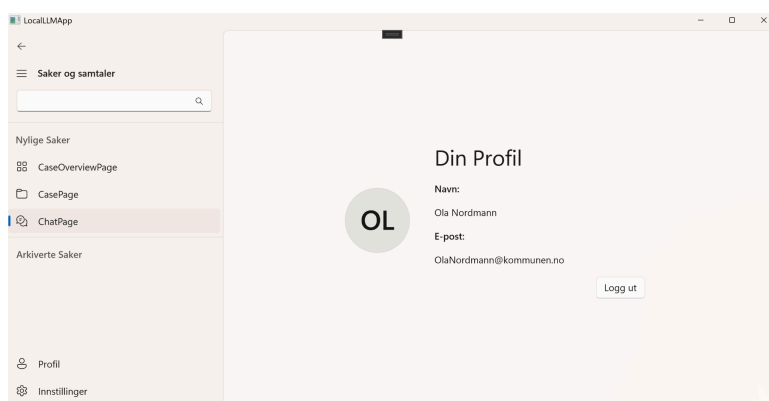
D4: Chat Page Prototype



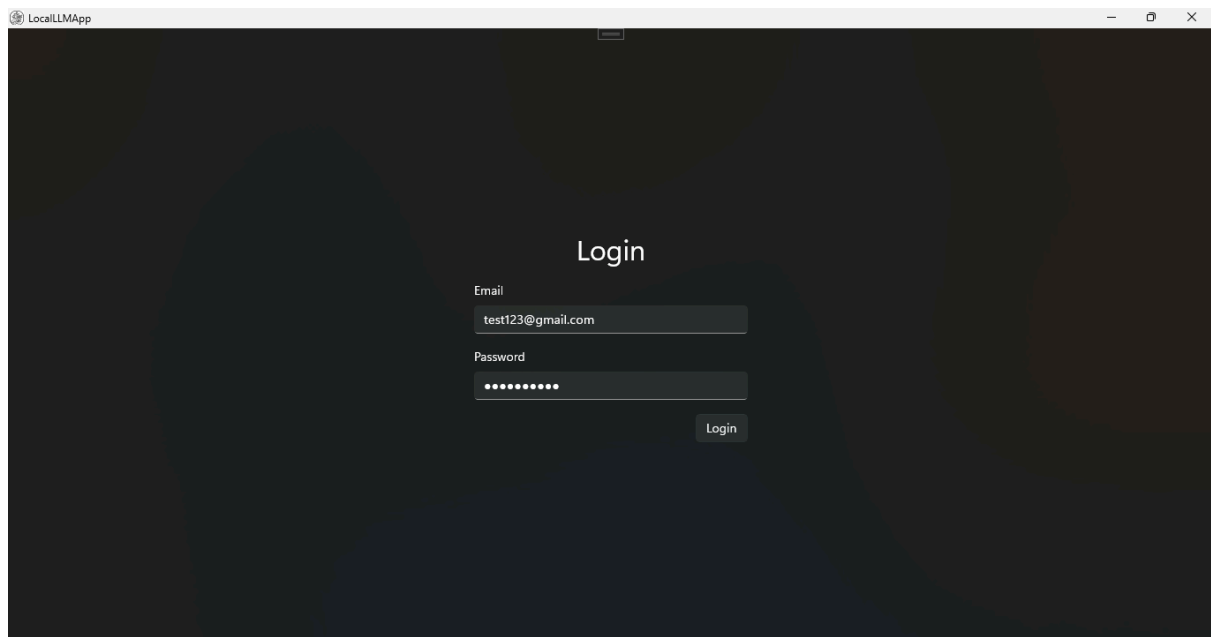
D5: Settings Page Prototype



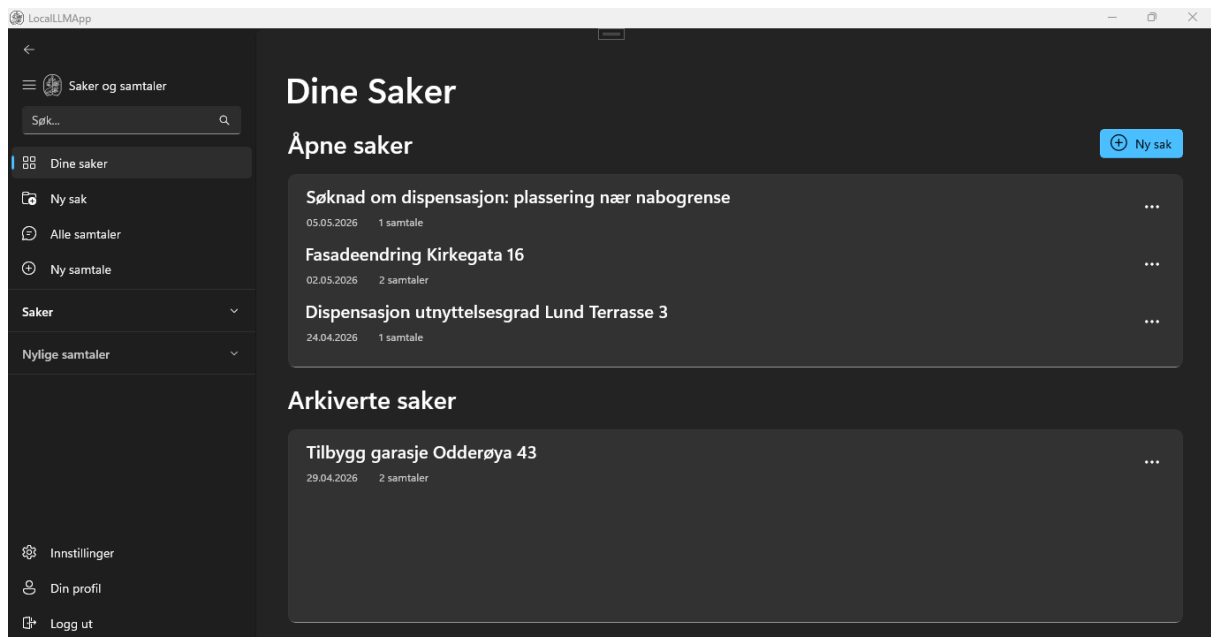
D6: Profile Page Prototype



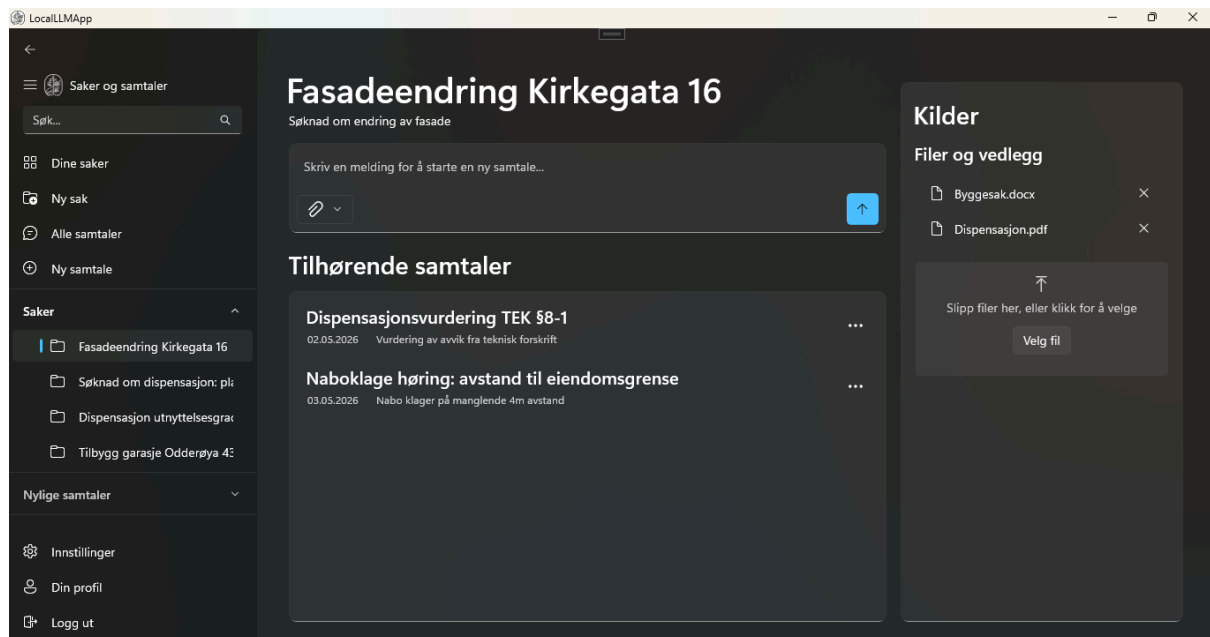
D7: Login Page Final



D8: Case Overview Final



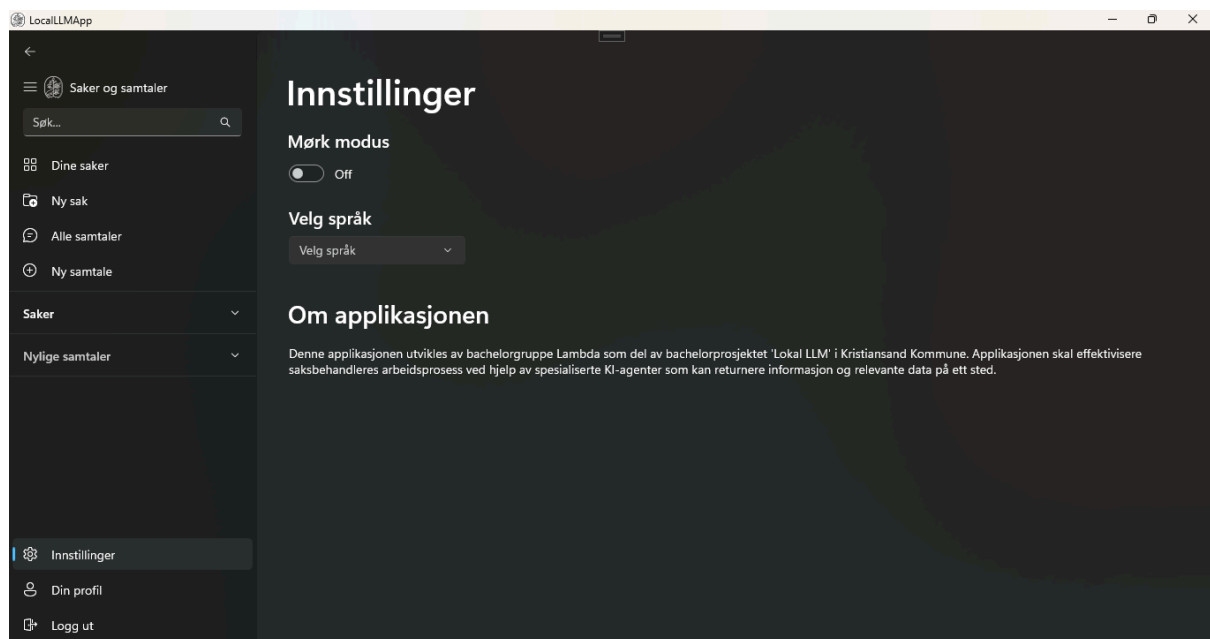
D9: Case Page Final



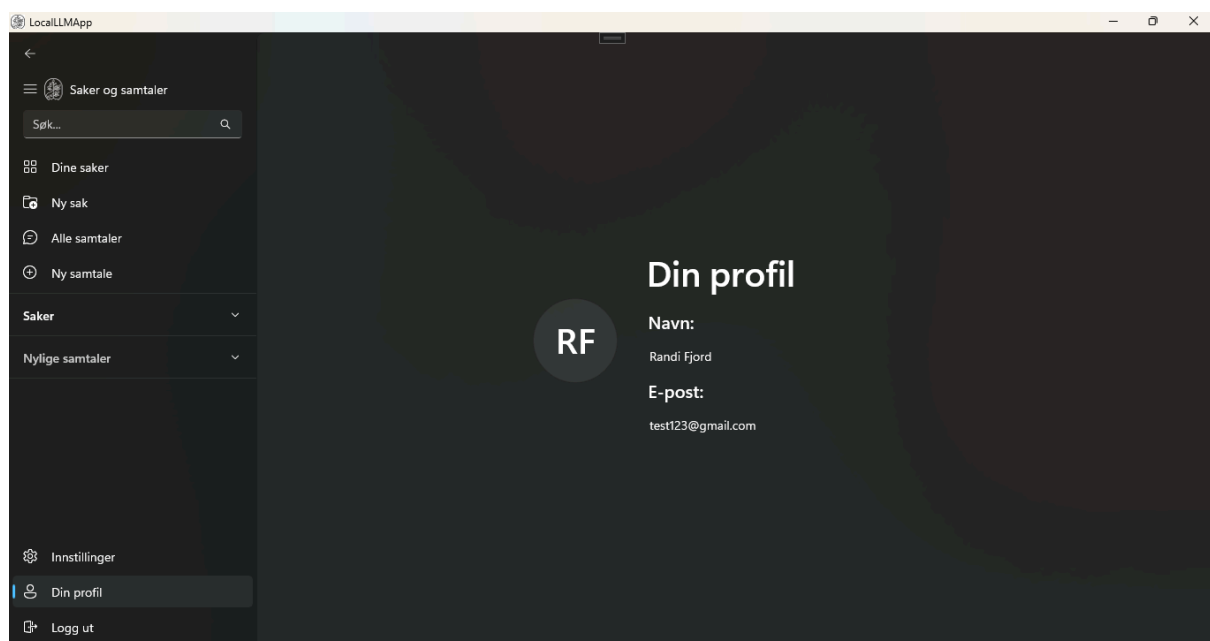
D10: Chat Page Final



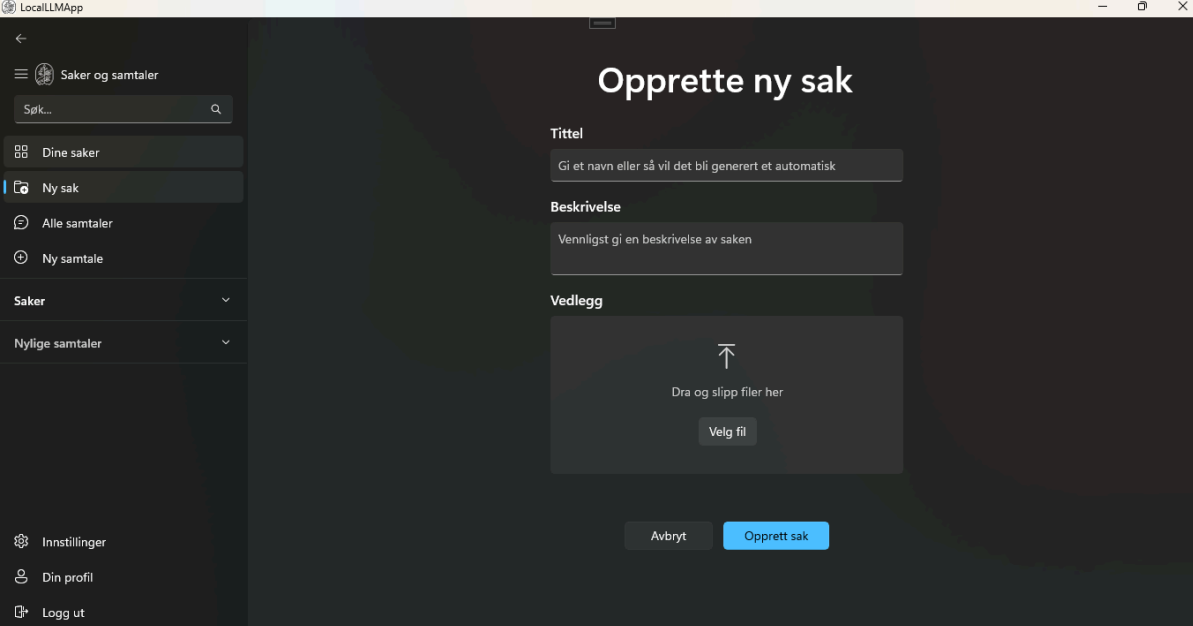
D11: Settings Page Final



D12: Profile Page Final



D13: Create Case Page Final



LocalLLMApp

←

Saker og samtaler

Søk...

Dine saker

Ny sak

Alle samtaler

Ny samtale

Saker

Nylige samtaler

Innstillinger

Din profil

Logg ut

Opprette ny sak

Tittel

Gi et navn eller så vil det bli generert et automatisk

Beskrivelse

Vennligst gi en beskrivelse av saken

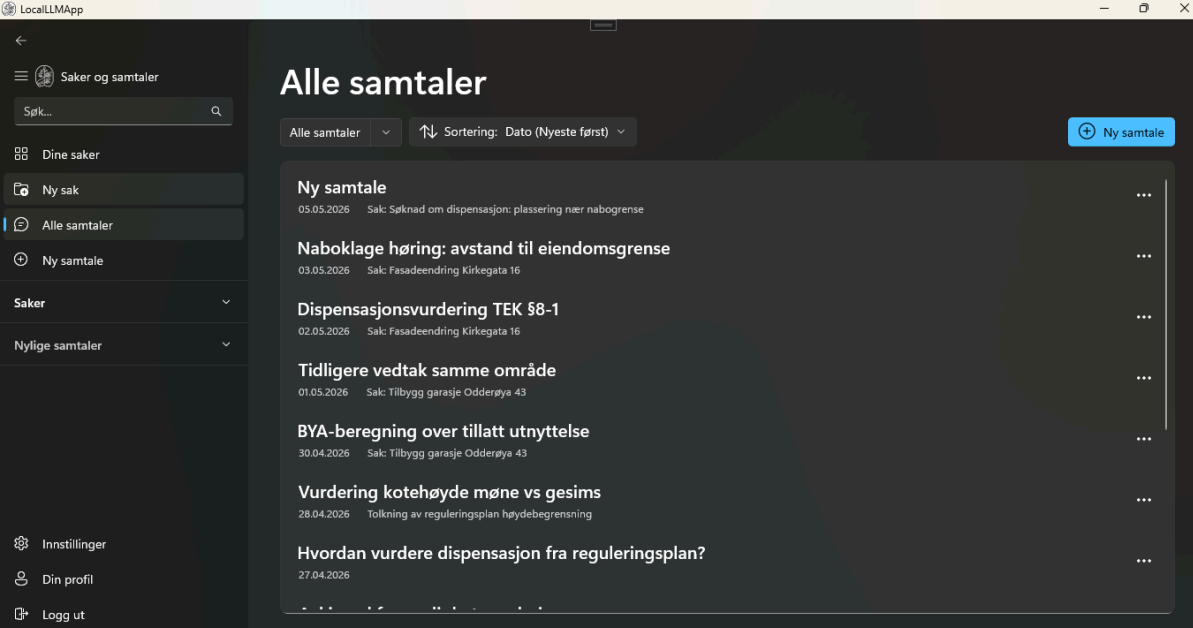
Vedlegg

Dra og slipp filer her

Velg fil

Avbryt Opprett sak

D14: Chat Overview Page Final



LocalLLMApp

←

Saker og samtaler

Søk...

Dine saker

Ny sak

Alle samtaler

Ny samtale

Saker

Nylige samtaler

Innstillinger

Din profil

Logg ut

Alle samtaler

Alle samtaler

Sortering: Dato (Nyeste først)

Ny samtale

Ny samtale

05.05.2026 Sak: Søknad om dispensasjon: plassering nær nabogrense

Naboklage høring: avstand til eiendomsgrænse

03.05.2026 Sak: Fasadeendring Kirkegata 16

Dispensasjonsvurdering TEK §8-1

02.05.2026 Sak: Fasadeendring Kirkegata 16

Tidligere vedtak samme område

01.05.2026 Sak: Tilbygg garasje Odderøya 43

BYA-beregning over tillatt utnyttelse

30.04.2026 Sak: Tilbygg garasje Odderøya 43

Vurdering kotehøyde møne vs gesims

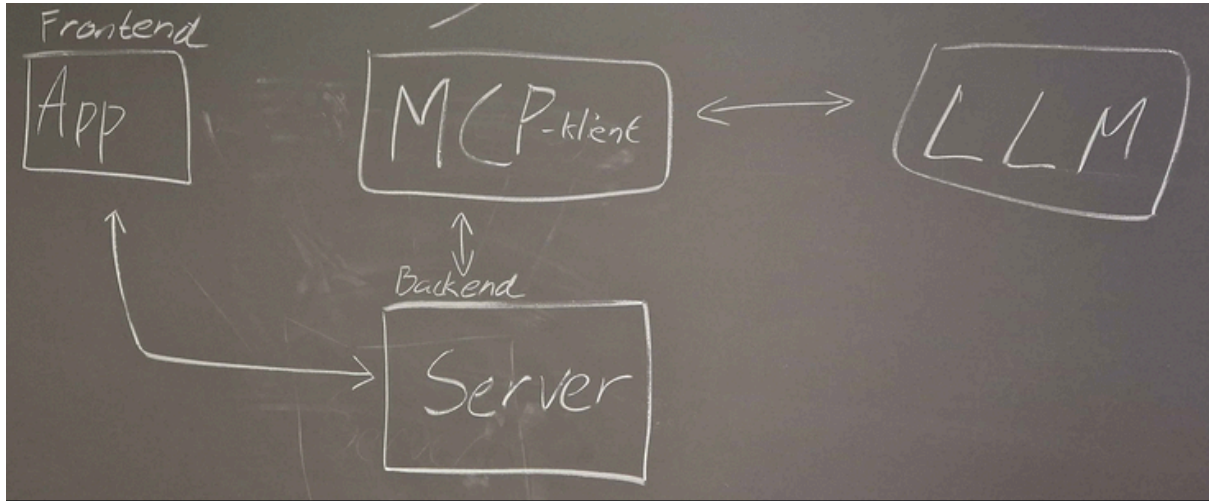
28.04.2026 Tolkning av reguleringsplan høydebegrensning

Hvordan vurdere dispensasjon fra reguleringsplan?

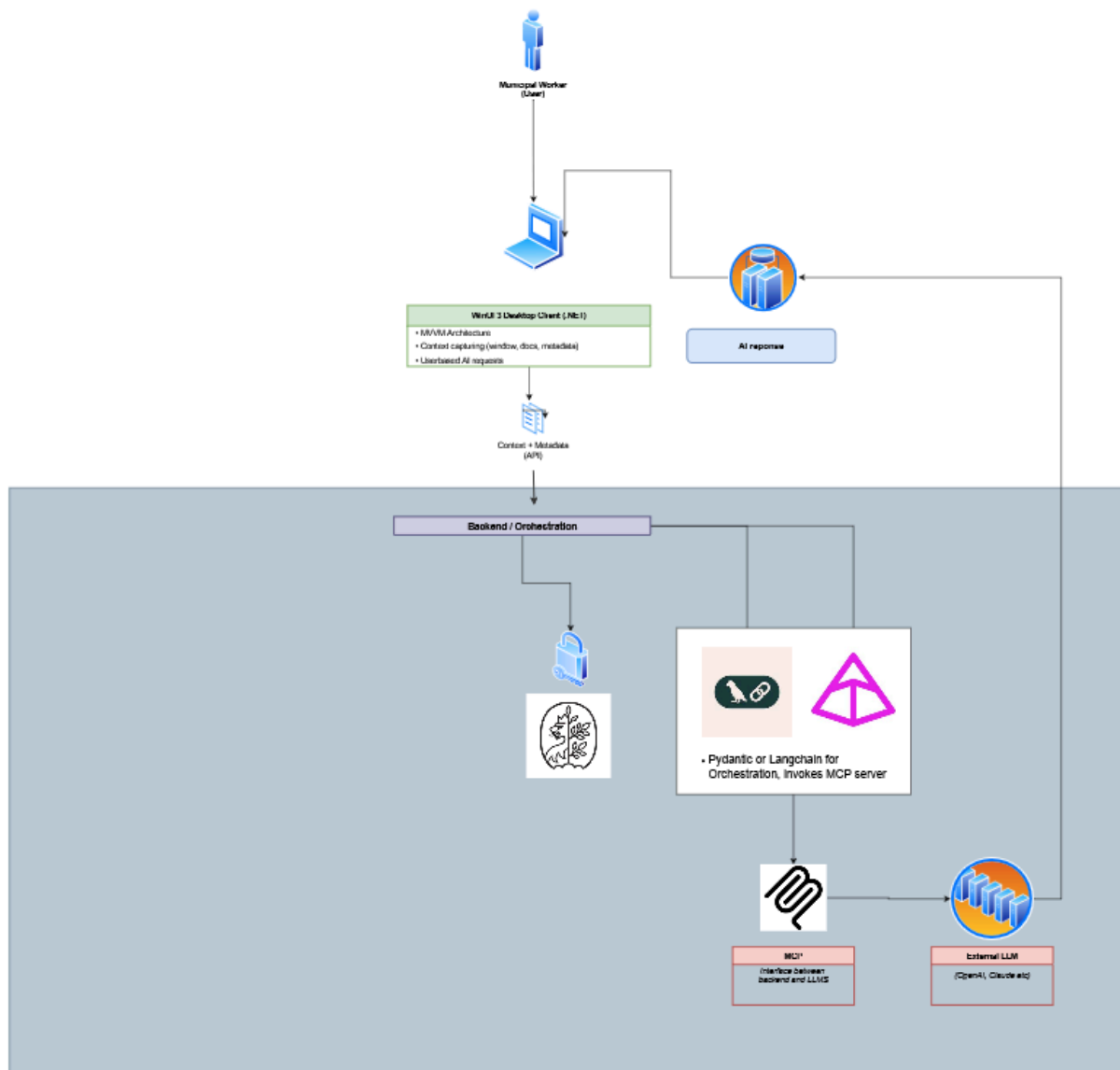
27.04.2026

Appendix E: System Architecture and Flow

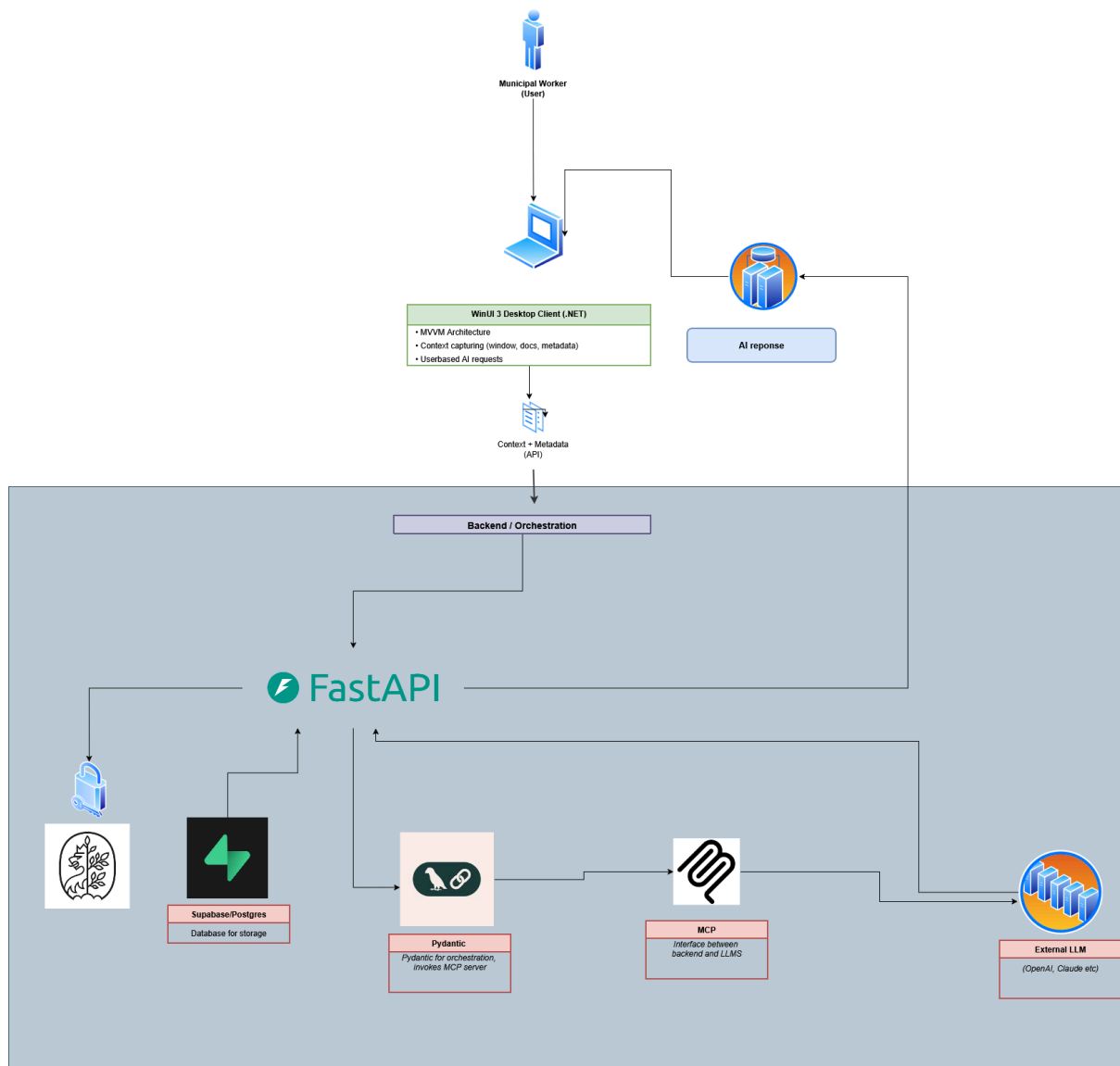
E1: First Iteration System Architecture



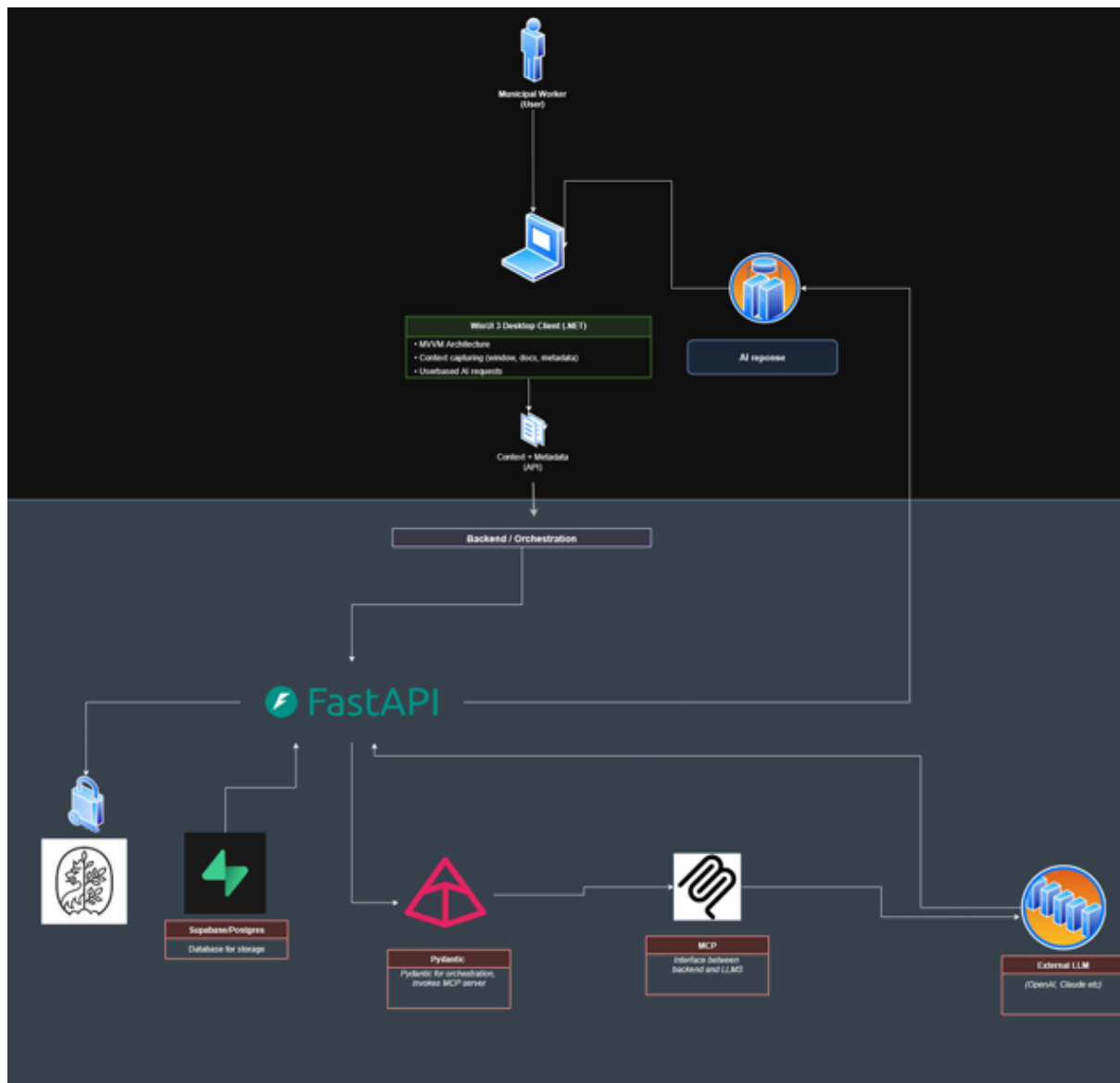
E2: Second Iteration System Architecture



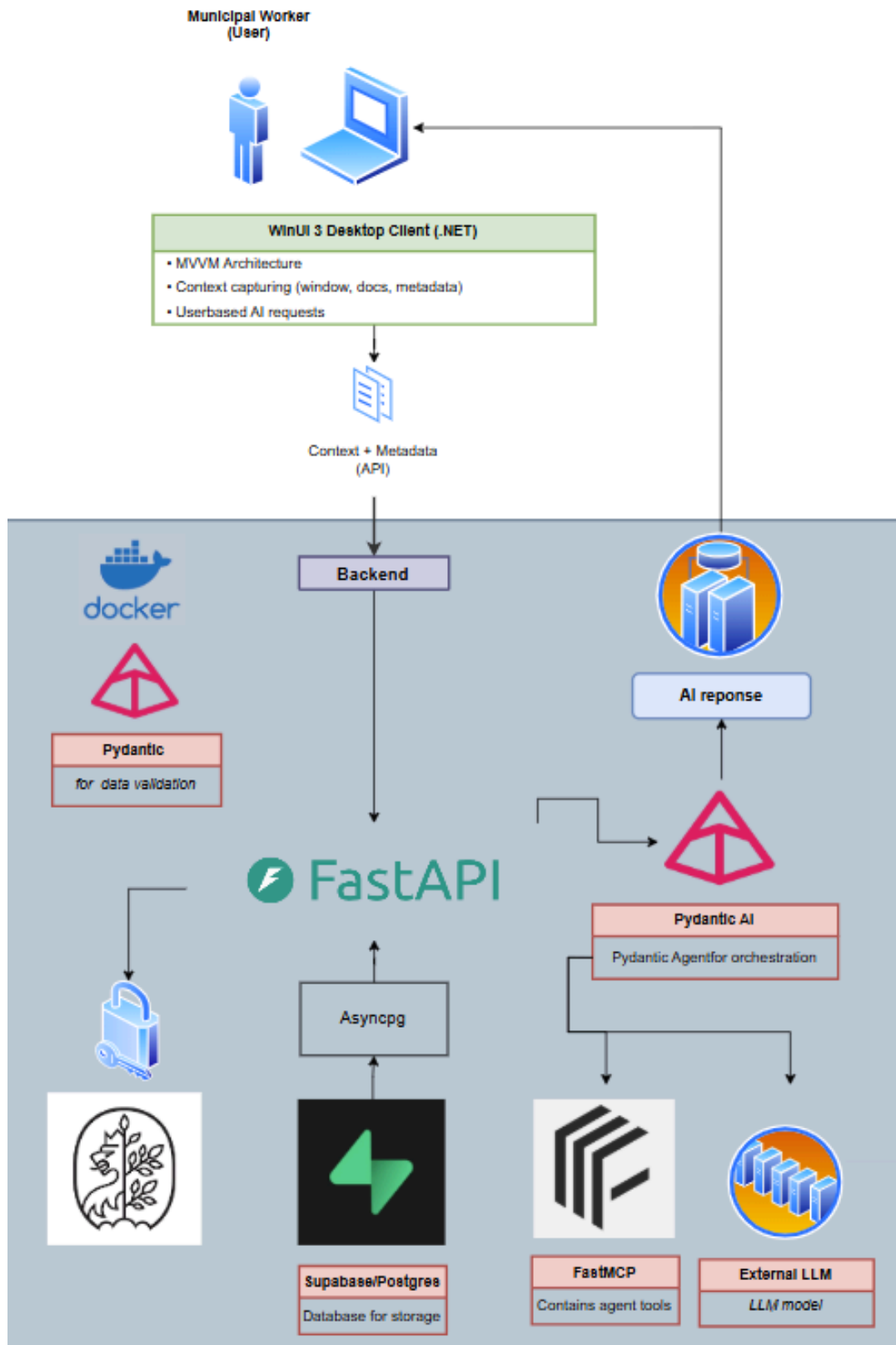
E3: Third Iteration System Architecture



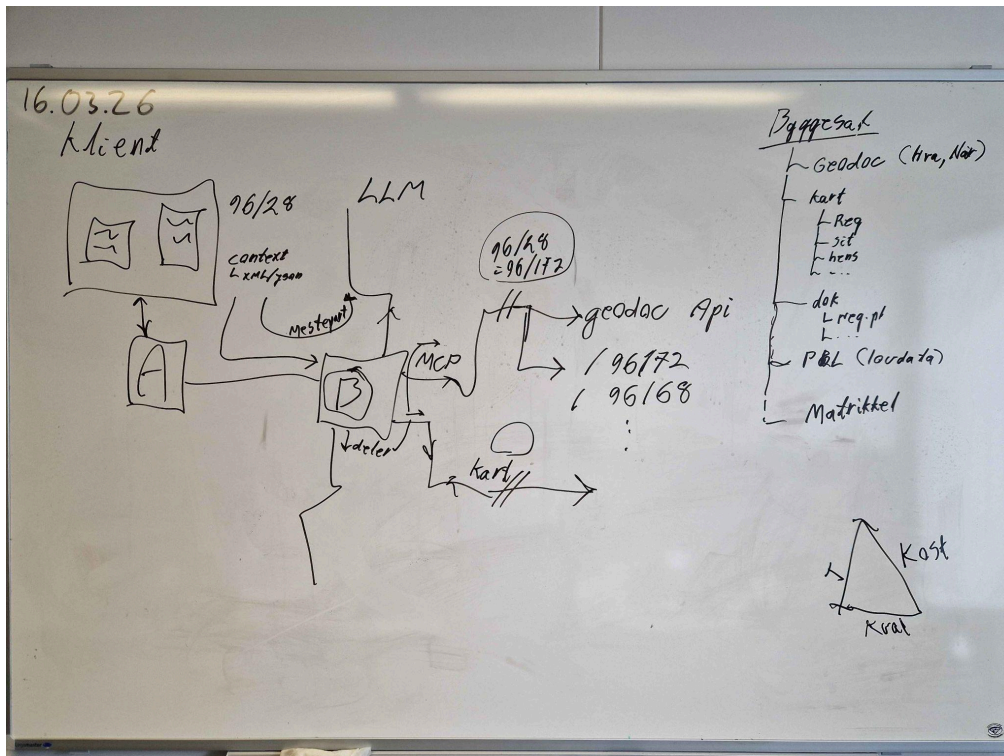
E4: Fourth Iteration System Architecture



E5: Fifth Iteration System Architecture



E6: Endpoint Architecture



E7: Sequence Diagram



Appendix F: Logging of Hours

F1: Timesheet PI 1

 Sprint 1	Stine	Kata	Jørgen	Frank	Jon	Sammenlagt
06/01						0
07/01						0
08/01						0
09/01						0
10/01						0
11/01						0
12/01						0
13/01						0
14/01						0
15/01						0
16/01						0
17/01						0
18/01						0
19/01						0
20/01						0
21/01						0
22/01						0
23/01						0
Totalt:	0	0	0	0	0	0

 Sprint 2	Stine	Kata	Jørgen	Frank	Jon	Sammenlagt
26/01	6	4	6	Sykdom	6	22
27/01	5	6.5	3	Sykdom	3	17.5
28/01	6	3	2	Sykdom	4	15
29/01	5	5	5	5	3	23
30/01	6	7.5	5.5	4.5	2.25	25.75
31/01	0	0	0	0	0	0
01/02	0	0	0	1	0	1
02/02	6	6	6	6	6	30
03/02	2	4.5	5	2	3	16.5
04/02	4.5	4.5	5	6	4	24
05/02	4	Jobb	Utilgjengelig	5	5	14
06/02	4	2	5.5	4.5	4.5	20.5
07/02	0	0	0	0	0	0
08/02	0	0	0	1	0	1
09/02	6	6	6	6	6	30
10/02	6	6	6	6	6	30
11/02	5.5	5.5	5.5	5.5	5.5	22
12/02	3	4	5.5	5	4	21.5
13/02	4.5	6	4	4	4	22.5
Totalt:	73.5	70.5	70	61.5	66.25	341.75

Review Week	Stine	Kata	Jørgen	Frank	Jon	Sammenlagt
16/02	6	6	2	6	6	26
17/02	7	5	2	2	3	19
18/02	7	6.5	4.5	6	6	30
19/02	7	7	2	5	6	27
20/02	4	4	4	4	4	20
21/02						
22/02						
Totalt:	31	28.5	14.5	23	25	122

F2: Timesheet PI 2

 Sprint 1	Stine	Kata	Jørgen	Frank	Jon	Sammenlagt
23/02	6	6	6	6	6	30
24/02	4	2	3	3.5	6	18.5
25/02	7	8.5	5	6	1	27.5
26/02	4.5	4	5	5.5	Gullmedalje i Student-NM for engvolleyball	19
27/02	5	5	3	5	5	23
28/02			0			0
01/03			3			3
Totalt uke 9	26.5	25.5	25	26	18	121
02/03	7	5	Syk	6	5	23
03/03	9	5.5	Syk	2.5	1	18
04/03	6	3	Syk	5	5	19
05/03	5	jobb	Syk	6	2	13
06/03	3	0.5	Syk	5	6	14.5
07/03	1					1
08/03	2					2
Totalt uke 10	33	14	0	24.5	19	90.5
09/03	2 (syk)	6.5	6	6	6	24.5
10/03	6	2	1	5	7	21
11/03	5	5	1	5.5	5	21.5
12/03	Syk	jobb	3	6	6	15
13/03	5	7	4	5	7	28
14/03	2					2
15/03	2		2			4
Totalt uke 11	20	20.5	17	27.5	31	116
Totalt Sprint 1:	79.5	60	42	78	68	327.5

 Sprint 2	Stine	Kata	Jørgen	Frank	Jon	Sammenlagt
16/03	6	6	5.5	6	6	29.5
17/03	6	8.5	3	4	5	26.5
18/03	6	5	1 (Syk)	6	5	22
19/03	5	5	Syk	6	1 (England)	16
20/03	4	Reise	2 (Syk)	8	England	12
21/03				2		2
22/03			1.5			1.5
23/03	6	6	6	Syk	England	18
24/03	4	5	2	7.5	3	21.5
25/03	6	5	3	7.5	4	25.5
26/03	4	1	4	6	6	21
27/03	5	5	1	6	5	22
28/03					Påske	0
29/03					Påske	0
30/03	3.5	1			Påske	4.5
31/03					Påske	0
01/04	1	5	1		Påske	7
02/04					Påske	0
03/04	2				Påske	2
04/04	5				Påske	5
05/04	2		5		Påske	7
06/04	6				Påske	6
07/04	7	2	4	2.5	5	20.5
08/04	7	6	6.5	6	5	30.5
09/04	6	6	3	5	5	25
10/04	6	6	5	6	6	29
Totalt:	97.5	72.5	50.5	78.5	55	354

Review Week	Stine	Kata	Jørgen	Frank	Jon	Sammenlagt
13/04	6	5	6	6	11	34
14/04	6	4	5.5	6	8	29.5
15/04	7	0	6.5	6	6.5	26
16/04	9	4	4.5	7	10	34.5
17/04	5	5	5	5	10.5	30.5
18/04			2			2
19/04			1			1
Totalt:	33	18	30.5	30	46	157.5

F3: Timesheet PI 3

Table7 ▾						
Sprint 1 ▾	Stine ▾	Kata ▾	Jørgen ▾	Frank ▾	Jon ▾	Sammenlagt ▾
20/04	6.5	7	7	6	7	33.5
21/04	6	9	8	6.5	11.5	41
22/04	6	6	6	7	6	31
23/04	7	5	10	7	6.5	35.5
24/04	6	8	7.5	7.5	5	34
25/04	4	0	0	0	0	4
26/04	0	0	0	1	0	1
27/04	5	5	5	3	6	24
28/04	5	3	0	3	7.5	18.5
29/04	8.5	8.5	0	3	7	27
30/04	7.5	7.5	3	3.5	4	25.5
01/05	7	6	4	4	7	28
02/05	3.5	2.5	0	0	0	6
03/05	0	2.5	0	0	8	10.5
Totalt:	72	70	50.5	51.5	75.5	319.5

Note: The PI 3 timesheet only includes hours registered during PI 3 Sprint 1. As the report was submitted before the remaining PI 3 work was completed, hours from later sprints are not represented in this appendix.

Appendix G: Self and Team Evaluation

G1: Self evaluation - Kata

Throughout the course of the project I've held the role of automation architect. In regards to system architecture, my main contributions have been within choosing and implementing the frontend (WinUI) and orchestration layer (Pydantic AI), though I've contributed with research and testing of functionality within the database, FastAPI and FastMCP server as well. My responsibilities in regards to the automation architect role is best displayed in my work with the system's orchestration layer, which I had sole responsibility for. I was responsible for researching alternative technologies and solutions, as well as their implementation into the system to enable the orchestration of user requests through LLM agents. My work with Pydantic AI also overlaps with the system's MCP, responsible for supplying the AI with tools and functionality. As a result of my work with Pydantic AI, I drove the shift from the initial MCP client/server and FastApiMCP solutions, to the final mounted FastMCP server architecture.

Within the team, I contributed with discussions of best practices and process improvements. Alongside the rest of the team, I've aided in structuring work and deadlines, especially in the second half of the project when the Scrum master role was split between the team members. I've also contributed significantly in the writing of the bachelors report.

G2: Self evaluation - Jon

At the beginning of the project, I was assigned the role of Security Engineer. Although security hasn't been the primary focus, I've tried to keep security considerations in mind throughout my work. In the early phases of the project when we did most of our initial field research, I was responsible for conducting the interviews that helped us form an understanding of the municipality's work and systems. I also proposed a lot of the routines for project management, including methods for logging work, tracking hours, and in general striving towards a more structured workflow. In addition, I picked up the role as Scrum Master around halfway through the term, and has given me the responsibility of overseeing how the team approaches the project. Ensuring that our work processes align with the project goals and helping maintain focus on the overall objectives. Additionally having the responsibility to resolve and mediate conflicts within the team.

From a technical perspective, I've worked actively on both the frontend and backend development, which has allowed me to act as a bridge between the two areas and maintain cohesion. I implemented most of the frontend functionality related to data processing and handling, as well as designing and establishing the data flow pipeline between the frontend and backend. In addition I also implemented the file handling and file storage functionalities. Having been involved in all aspects of the application has helped me maintain an overall perspective on the project's progression, which in turn has assisted my work as Scrum Master.

G3: Self evaluation - Stine

On the UX side, I was responsible for planning and conducting user interviews, structuring insights, and translating findings into user journeys, sketches and wireframes. I held two of the user interviews after the initial ones, and conducted several usability tests with caseworkers involved in the project. To bridge design and implementation, I mapped WinUI 3 components directly to wireframe elements, producing a reference document that was iterated across the project to serve as a shared implementation guide for the team. On the technical side, I was responsible for the frontend implementation, building and refining UI components and establishing a consistent visual standard across the application. I also contributed to collaboration between frontend and backend by participating in data modelling discussions to ensure a shared understanding across the team.

Within the team, I took initiative in improving team processes. This included proposing structured summaries of work before group discussions to improve knowledge sharing. I also contributed to the standardization of backlog issue templates, and suggested a more iterative approach to sprint planning where initial high-level planning is refined after research, rather than fully defined upfront. Beyond my defined roles, I shared responsibility for external communication with our supervisor and other external parties, and contributed across areas when the project required it.

G4: Self evaluation - Jørgen

My main role initially throughout the first half of the project, was being the Scrum Master. In this role I have facilitated room for discussion and conversation. The role has included keeping focus on the project's direction, and reviewing sprints as well. Partaking in

discussion with the other team's scrum masters in a meeting facilitated by the client were done. These meetings had a goal to share, discuss and improve on issues for each team. Facilitating words from the client to the team members, and vice versa, was also a part of my role. The second half of the project, the role was split up due to me having health issues. My role has instead taken the shape of being a supporting office administrative role. Sending emails, upkeep the team's log, writing on the rapport and scheduling meetings. Most of these tasks have been shared, whereas I have tried to take the initiative to perform these. I have also actively partaken in discussion within the team for the project's direction, assisting with knowledge for decisions along the way. Even though my capacity has been lower than I have wanted, I have still tried to contribute in the best way I can. Lastly it is important to note that even though my health has prohibited me from full capacity, I have gotten a role from the client to research other matters that will generate extra value for both the project and the company.

G5: Self evaluation - Frank

My main roles throughout the project have been Release Manager and Co-Admin. Throughout the project, my main contribution was helping shape and maintain the architecture of the system. I have worked in multiple areas, mainly in the backend, implementing CI/CD pipeline, setup the database connection and authorization, as well as template queries. Furthermore, I have worked on parts of the context pipeline, implementing the encoding for screenshots. In addition to development work, I have contributed to discussions and provided technical insight into key decisions.

Furthermore, I took on a coordinating role as Co-Admin that spans across the different teams in the FoU program, laying a foundation and providing guidance for the shared infrastructure utilized by the different teams. This role has been a challenging experience, as it required balancing the role as a system administrator alongside being a developer within my own team. This meant I had to prioritise my tasks and accept I could not always contribute equally in both roles. At the same time, the roles complemented each other and gave me a broader insight into the FoU program. Overall, my contribution is not limited to a single area, but rather focused on supporting the project in its entirety.

G6: Team reflection

During the bachelor's project, the team members have continuously worked to improve teamwork to drive the project forward. As noted during sprint and PI reviews, the team's strengths include strong communication and open dialogue, as well as focus on continuous improvement of internal processes. The team members share common values, with the most significant being honesty, respect and responsibility. These values were discussed and agreed upon in our group contract, and have been consistently upheld in the team's work throughout the course of the project. Preserving these values has enabled the team to overcome hurdles within the project execution.

Appendix H: The Client's Evaluation

Plan & Bygg, Kristiansand Kommune (FoU Bachelorprosjekt 2026)



Team Lambda – Lokal LLM

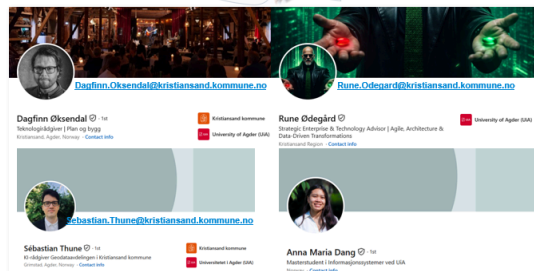
Statement from Kristiansand Municipality

Team Lambda has, over the course of this project, demonstrated a notable transition from operating as students to acting as capable consultants. The team has shown a clear understanding of the client's target state and has delivered an MVP that is not only functional, but also structured in a way that can realistically be further developed towards production, with potential applicability beyond the local context.

The work carried out by the team establishes an important architectural foundation for how municipalities can approach the rapid adoption of AI in the public sector. In particular, the solution reflects a conscious balance between enabling advanced AI capabilities while maintaining control over data, infrastructure, and usage. The approach supports the possibility of introducing specialized and locally governed AI assistants across different domains and departments within a municipality.

In addition to their technical delivery, the team has demonstrated a high level of professionalism throughout the project. They have handled a demanding and at times challenging project period with maturity, mutual support, and a strong sense of shared responsibility. Their ability to take ownership not only of the solution, but also of each other's challenges, reflects a level of teamwork that stands out.

Overall, the team's contribution is considered both valuable and forward-looking, and provides a solid foundation for continued development within this domain.



Appendix I: Group Contract

DEL 1

Gruppekontrakt for studentgrupper – Generell del

Gruppemedlemmer:

Frank Hovet

Jørgen Ege

Jon Engravslia Aarebakk

Stine Strand

Kata-Loore Tamm

Varighet (semester og år): Semester 6, Vår 2026

Emner som inngår i kontrakten: IS-314

Om gruppekontrakten

Gruppekontrakten består av to deler: Den generelle delen (Del 1) beskriver de overordnede ansvarsområder, plikter og prosesser for gruppearbeidet. Del 2 beskriver gruppenes individuelle målsettinger, arbeidsformer og tilpassede spilleregler. Ved motsetninger/konflikter mellom den generelle delen (Del 1) og den individuelle delen (Del 2), **vil alltid den generelle delen av kontrakten (Del 1) være gjeldende.**

Hensikt med gruppekontrakt

Gruppearbeid er en helt sentral del av læringsaktivitetene i våre studier. Dette er tett knyttet til læringsmålene i mange av våre emner, som i stor grad krever at studentene lærer seg å arbeide sammen med andre.

Arbeid i grupper er også viktig for å forberede alle på arbeidslivet etter studiene. De aller fleste av våre studenter kommer til å arbeide i team eller på andre måter måtte samarbeide med andre, og alle arbeidsgivere vil være interessert i kandidater som har god erfaring med dette. I arbeidslivet vil det sjeldent være slik at man kan velge hvem man arbeider sammen med. Slik er det også ofte hos oss, siden vi mener det er viktig at alle får god kompetanse i å arbeide sammen med forskjellige personer med ulike kvaliteter og interesser.

Gruppearbeid er også viktig for den sosiale tilhørigheten for våre studenter. Det kan være ganske ensomt å være student. Vi er overbevist om at det derfor er viktig og riktig å ha en tilhørighet til en mindre gruppe som er avhengige av at alle bidrar.

Den enkeltes ansvar og plikter

- Alle medlemmer i en studentgruppe skal bidra til et felles produkt (jfr. [§29 Forskrift om studier og eksamen ved UiA](#)).
- Alle medlemmer lytter til hverandre og er åpne for innspill.
- Alle medlemmer skal overholde frister satt av gruppen selv og de som er satt i emnet.
- Alle medlemmer er forventet å stille opp i henhold til avtalt møtetidspunkt.
- Ved sykdom eller andre uforutsette forhold som forhindrer oppmøte skal dette meldes skriftlig til øvrige gruppemedlemmer så snart som mulig.

Konflikthåndtering

Konflikter er ikke nødvendigvis noe negativt – det blir først et problem når det blir dysfunksjonelt. Konflikt kan oppstå på grunn av ulike synspunkter, arbeidsstiler eller interesser. Slike forskjeller kan ofte føre til nye og gode ideer – så lenge de håndteres riktig. Når gruppen håndterer konflikter, skal det **fokuseres på sak, ikke person**. Som hovedregel, skal man søke å løse konflikter på lavest mulig nivå (innad i gruppen). Følgende gjelder ved konflikter som ikke kan løses i gruppen:

- **Gruppen har ikke anledning** til å ekskludere gruppemedlemmer eller splitte/oppløse grupper uten at instituttet er involvert.

- En kortfattet utredelse av konflikten skal sendes skriftlig til emneansvarlig, snarest mulig og senest innen en måned før eksamen/eksamensinnlevering.
- Emneansvarlig og/eller studieprogramleder vil innkalle hele gruppen til samtale som legger grunnlaget for finne løsninger på konflikten, evt. vurdering av eksklusjon av enkeltmedlemmer eller oppsplitting/oppløsning av grupper. Alle medlemmer i gruppen skal stille til dette møtet.

DEL 1

Ved manglende bidrag fra enkeltstudenter

Ved manglende bidrag fra enkeltstudenter, har gruppen anledning til å unnlate å føre opp navn på medstudenter i gruppen ved innleveringer, hvis ikke disse har bidratt i henhold til avtale. Følgende vil gjelde i slike situasjoner:

- **Studenter som er i faresonen** for å ikke få påført sitt navn på et felles gruppeprodukt skal få et skriftlig varsel om dette fra resten av gruppen, med mulighet for å ta igjen sin del av arbeidet. Varaset skal sendes i god tid før innleveringsdato.
- Hvis ikke varaset fører til endringer i bidraget, er det et krav om at studenter som ikke får sitt navn påført et gruppearbeid skal få en skriftlig begrunnelse om dette fra de øvrige gruppe medlemmene.
- Kopi av varsler og begrunnelser skal sendes til emneansvarlig.
- Om studenters navn blir utelatt på en innlevering så har disse studentene rett til å fremvise dokumentasjon på det arbeidet de har gjort og bidratt med i gruppearbeidet. I slike tilfeller er det instituttet ved emneansvarlig og studieprogramleder som foretar en faglig vurdering av om dette arbeidet kan anses som tilstrekkelig for å godkjennes som del av felles produkt (jfr. [§29 Forskrift om studier og eksamen ved UiA](#)).

Vi bekrefter herved å ha gjort oss kjent med de retningslinjer som gjelder for gruppesamarbeid og forplikter oss med dette til å følge opp vår del av plikter og ansvar ved arbeid i grupper som beskrevet her i Del 1 av gruppekontrakten.

(Studentenes navn, dato, og signatur):

Jørgen Ege

Jon Engravslia Aarebakk

Stine Strand

Kata-Loore Tamm

Frank Hovet

12.01.2026

Gruppekontrakt for studentgrupper – Individuell del

Hensikten med den individuelle delen av gruppekontrakten er at gruppen kommer til en felles enighet om gruppens målsetting, håndtering av individuelle hensyn, forventninger, arbeidsform, kommunikasjon og spilleregler i gruppearbeidet.

Utarbeidelse av individuell del av gruppekontrakten

Å jobbe i gruppe gjenspeiler hva som skjer på arbeidsplassen. Som med alle nye team, komiteer eller arbeidsgrupper, medfører det utfordringer. Derfor er det viktig å ta dette på alvor.

Siden du har blitt tildelt en gruppe der din deltakelse kreves i ukene som kommer, må du først reflektere individuelt – og deretter kollektivt – over hvordan dere skal samarbeide i denne perioden. Følgende øvelse er laget for å hjelpe gruppen din med å møte disse kravene. Det anbefales sterkt at denne øvelsen gjennomføres i løpet av ett gruppemøte i et rolig og privat miljø som egner seg for målrettet arbeid – og litt moro!

1. Bli kjent

Finn en partner i gruppen din. Bruk 5–10 minutter på å snakke sammen og bli kjent. Noter ned personens navn, interesser, spesielle ferdigheter, arbeidserfaring (hvis noen), og annen nyttig informasjon. Gjenta så dette med en annen person, og fortsett til du har snakket med alle på gruppen.

2. Personlig bruksanvisning

I gruppearbeid møter man andre mennesker som har sine unike styrker og svakheter. Som enkeltperson skal du reflektere på følgende:

- Hva er mine egenopplevde styrker i samarbeidssituasjoner?
- Hva er mine egenopplevde svakheter i samarbeidssituasjoner?
- Hva trenger jeg fra gruppen for å være på mitt beste?

Presenter deretter refleksjonene enkeltvis i gruppen og diskutere sammen hvordan man kan ta hensyn til medlemmenes styrker og svakheter slik at gruppen presterer best mulig.

3. Forstå individuelle prioriteringer

Som enkeltperson, bruk diagrammet nedenfor til å markere et punkt mellom 0 og 100 som viser hvor viktig gruppearbeidet er for deg, sammenlignet med andre forpliktelser. For eksempel, du kan mene at gruppen har lav prioritet (f.eks. 10) fordi du har mange andre oppgaver. Eller, du kan mene at gruppen er svært viktig og gi det en høy prioritet (f.eks. 90).

0 50 100
ingen prioritet høy prioritet

Del nå dine prioriteringer med gruppen. Hver enkelt bør forklare hvorfor de har valgt som de har gjort og diskutere hva det betyr for samarbeidet fremover.

4. Del forventninger

Tenk gjennom gruppen, dets fremtid og din rolle i det. Du kan bruke følgende spørsmål som utgangspunkt:

- Hva ønsker du å oppnå som en del av gruppen?
- Hva bekymrer deg med gruppen?
- Hva mener du gruppen bør gjøre for å sikre best mulig resultat?

Del dine svar med resten av gruppen. Vær tydelig når du uttrykker deg, og lytt nøye til hva de andre sier. Sørg for at alle deltar, og at dere lærer så mye som mulig om hverandres syn på det kommende teamarbeidet.

5. Gruppens mål

Nå som dere har blitt kjent og delt litt personlig informasjon, er det på tide å avklare gruppens felles målsetting. Hva mener du målet for gruppen bør være, med tanke på de oppgavene dere er blitt tildelt? Del ditt forslag med

gruppen. Deretter skal gruppen utarbeide én samlet målformulering som alle kan ha eierskap til. Dette målet bør bli en samlende drivkraft i arbeidet fremover.

DEL 2

6. Spilleregler

Konflikter er ikke nødvendigvis noe negativt – det blir først et problem når det blir dysfunksjonelt. Konflikt kan oppstå på grunn av ulike synspunkter, arbeidsstiler eller interesser. Slike forskjeller kan ofte føre til nye og gode ideer – så lenge de håndteres riktig. Når dere håndterer konflikter, husk å fokusere på sak, ikke på person. Vær oppmerksom på at spillereglene som gruppen blir enige om må følge de overordnede reglene for konflikthåndtering beskrevet i Del 1 av gruppekontrakten. **Ved motsetninger/konflikter, gjelder regler for konflikthåndtering som beskrevet i Del 1.**

For å bli enige om spillereglene, kan følgende spørsmål diskuteres sammen:

- Hva er viktig for meg for å få til et godt samarbeid?
- Hvordan skal vi sikre at vi er trygge på hverandre?
- Hvordan skal vi sikre at alle tør å si ifra?
- Hvordan skal vi håndtere uenighet i gruppen?

7. Arbeids- og kommunikasjonsform

Gruppen skal utvikle og bli enige om hvordan arbeidet skal organiseres for å lykkes med oppgavene og målene som gruppen har satt. Det anbefales at overvekten av møter er fysiske der alle er til stede. I tillegg kan det etableres andre kommunikasjonskanaler (f.eks. telefon, Discord, Facebook, Google Drive, osv.). Arbeidsform og kommunikasjonskanalene dere definerer er avgjørende for å kunne levere felles gruppeoppgaver.

For å bli enige om arbeids- og kommunikasjonsform, kan følgende spørsmål diskuteres:

- Hvordan man skal man arbeide sammen og individuelt
- Når man skal arbeide sammen og individuelt. Hvilke møtepunkt trenger vi?
- Hvordan skal man kommunisere med hverandre
- Hvordan tar vi avgjørelser i gruppen? Leder, majoritet, enstemmig? Hvilke avgjørelser må hele gruppen være involvert i?

Gruppekontrakt – individuell del

Med utgangspunkt oppgaven over, skal gruppen komme frem til felles enighet om målsettinger, arbeidsform, forventninger osv. Dette skal beskrives her:

Gruppemedlemmer:
Jørgen Ege

Jon Engravslia Aarebakk

Stine Strand

Kata-Loore Tamm

Frank Hovet

Varighet (semester og år): Semester 6, Vår 2026

Emner som inngår i kontrakten: IS-314

Formell målformulering for gruppen:

Hele gruppen ønsker å gjennomføre et godt prosjekt, levere et produkt av høy kvalitet og oppnå så god karakter som mulig.

Personlig bruksanvisning (hvordan hensynta individuelle styrker og svakheter best):

Vi har erfaring innad i gruppa med å arbeide sammen. På bakgrunn av dette, kjenner vi hverandre bra nok til å hensynta individuelle styrker og svakheter.

DEL 2

Avtalt forventning til hverandre:

Forventningene vi har til hverandre, er at vi jobber mot et prosjekt med bra kvalitet som sikter mot Karakter A. Innad i gruppa forventer vi at alle overholder ansvar som man har blitt enige om.

Beskrivelse av arbeidsform og kommunikasjon:

Benytter SCRUM-metodikk. Kommunikasjon skjer over Discord.

Felles spilleregler:

Møtes til avtalt tid (med mindre beskjed om annet blir avklart).

God kommunikasjon - Si ifra tidlig angående problemer eller annet som er relevant innenfor gruppearbeid / prosjekt.

Respekt ovenfor hverandre.

Vi bekrefter herved at den individuelle delen av gruppekontrakten er utarbeidet i fellesskap og forplikter oss forplikter oss med dette til å følge opp den individuelle delen av gruppekontrakten.

(Studentenes navn, dato, og signatur):

Jørgen Ege

Jon Engravslia Aarebakk

Stine Strand

Kata-Loore Tamm

Frank Hovet

12.01.2026